

# Multi-Modal Deep Learning

Lecture Series | Summer 2025

**Institute for AI and Informatics in Medicine**

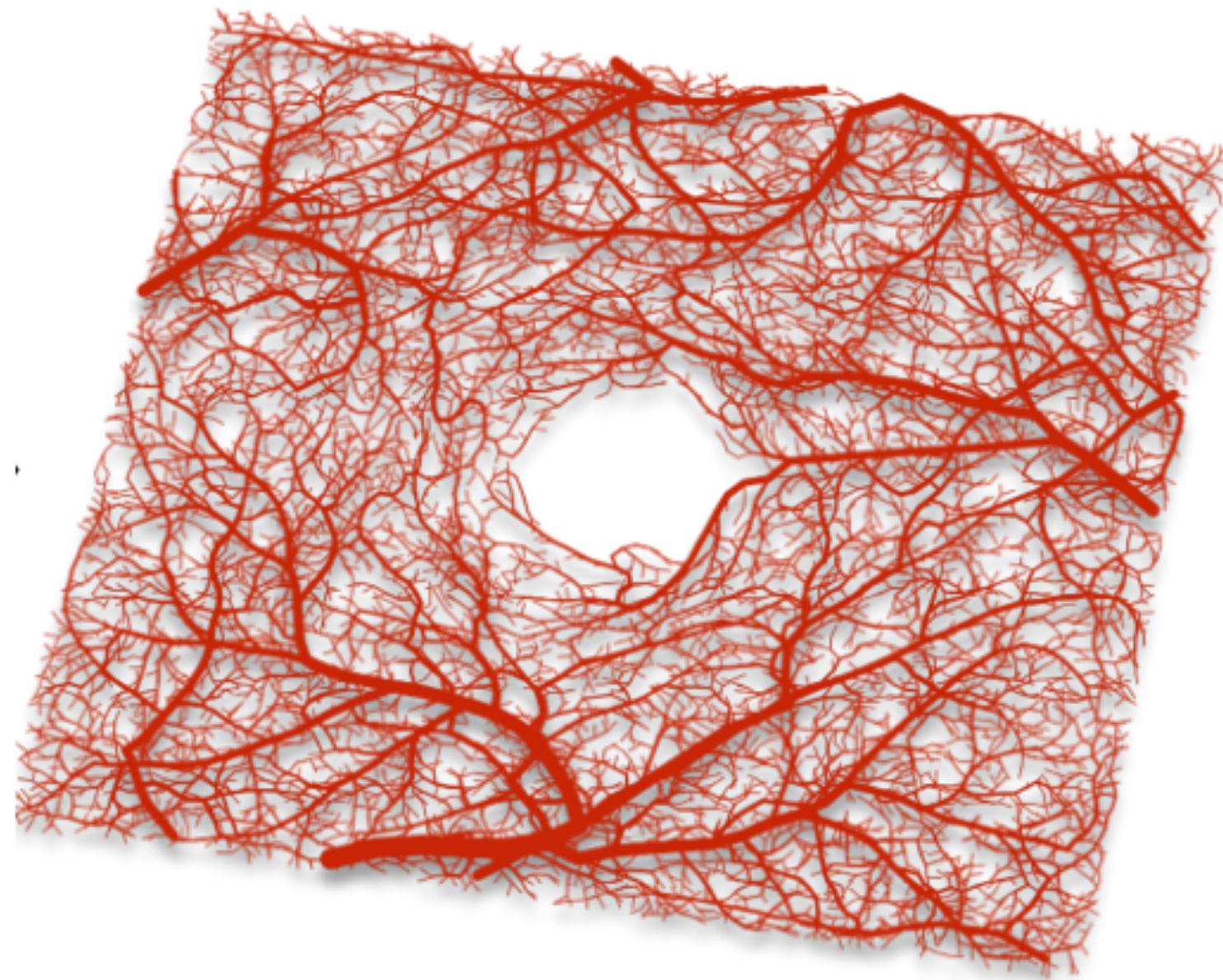
03.06.2025

# GRAPHS

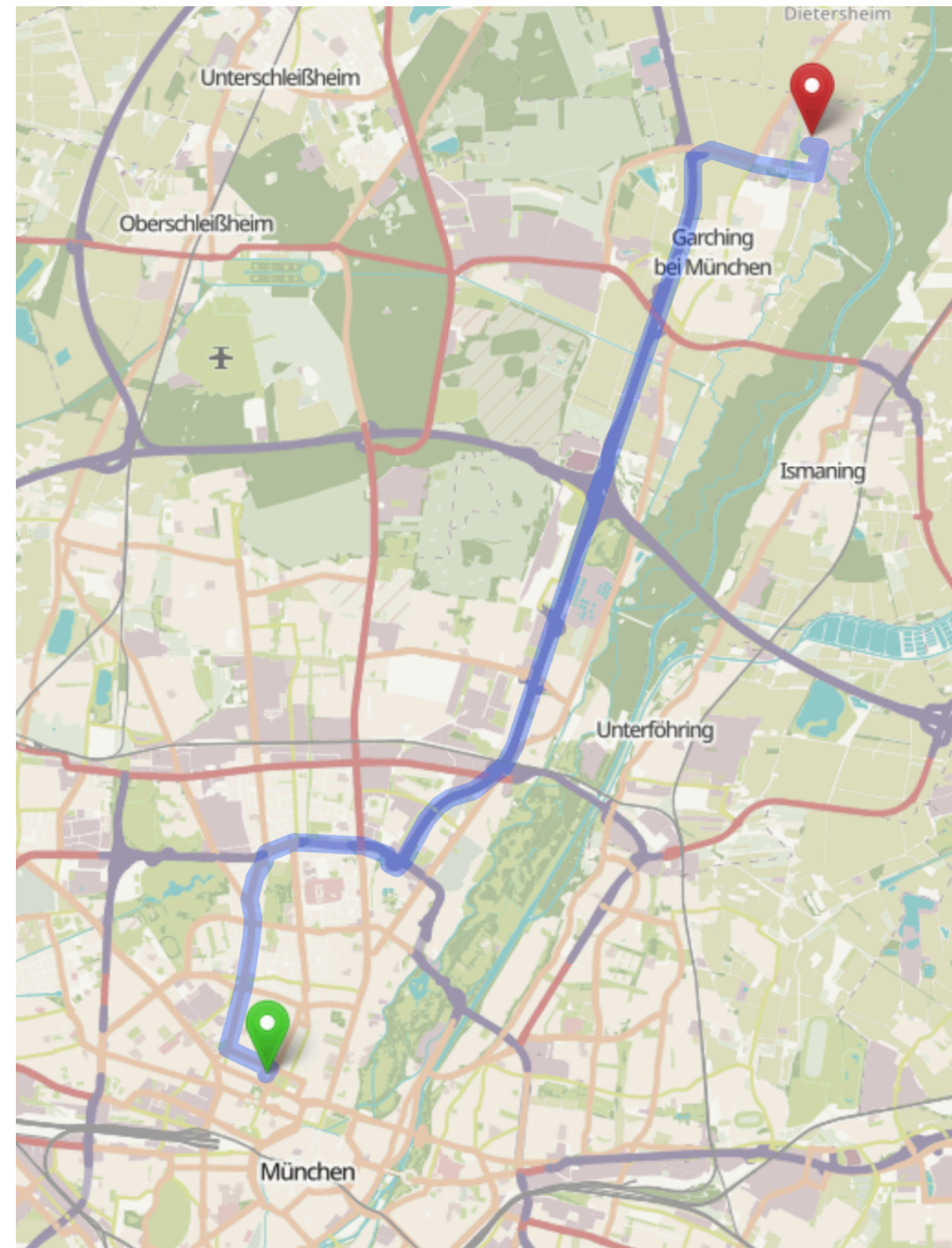
## Definition from Wikipedia

*"A graph is a structure consisting of **a set of objects** where some pairs of the objects are in some sense 'related'. The objects are represented by abstractions called vertices and each of the related pairs of vertices is called an edge."*

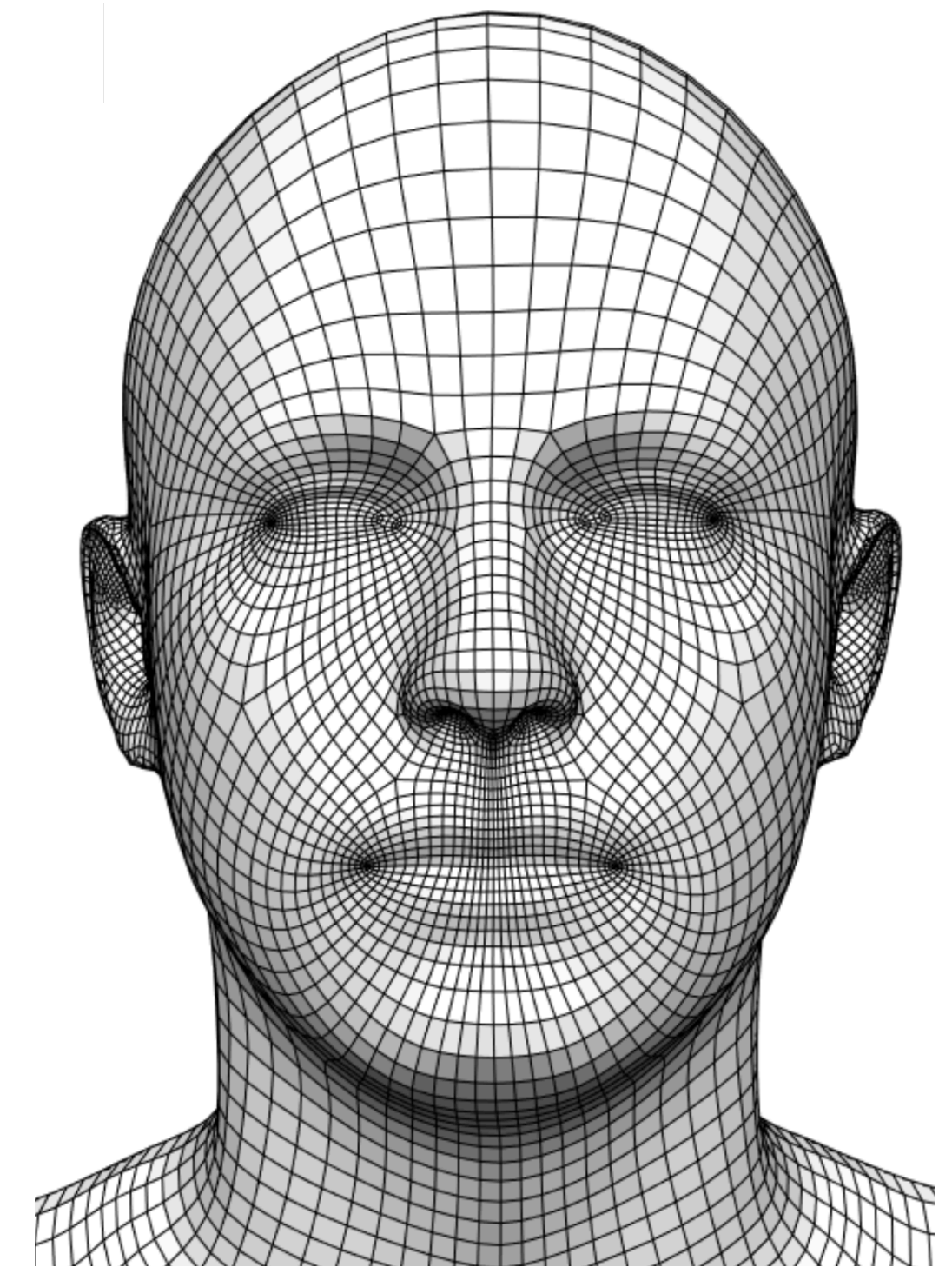
# DATA AS GRAPHS



**Blood vessel graph**

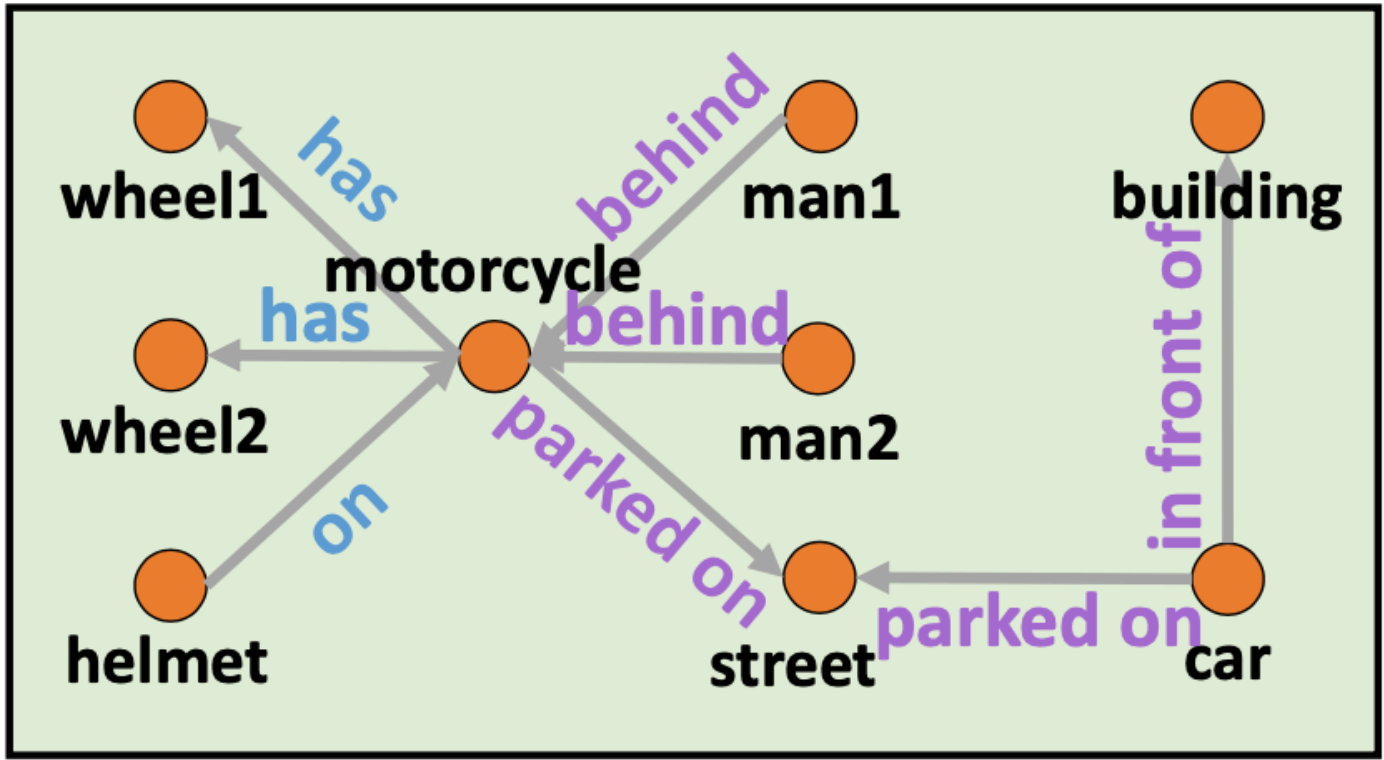
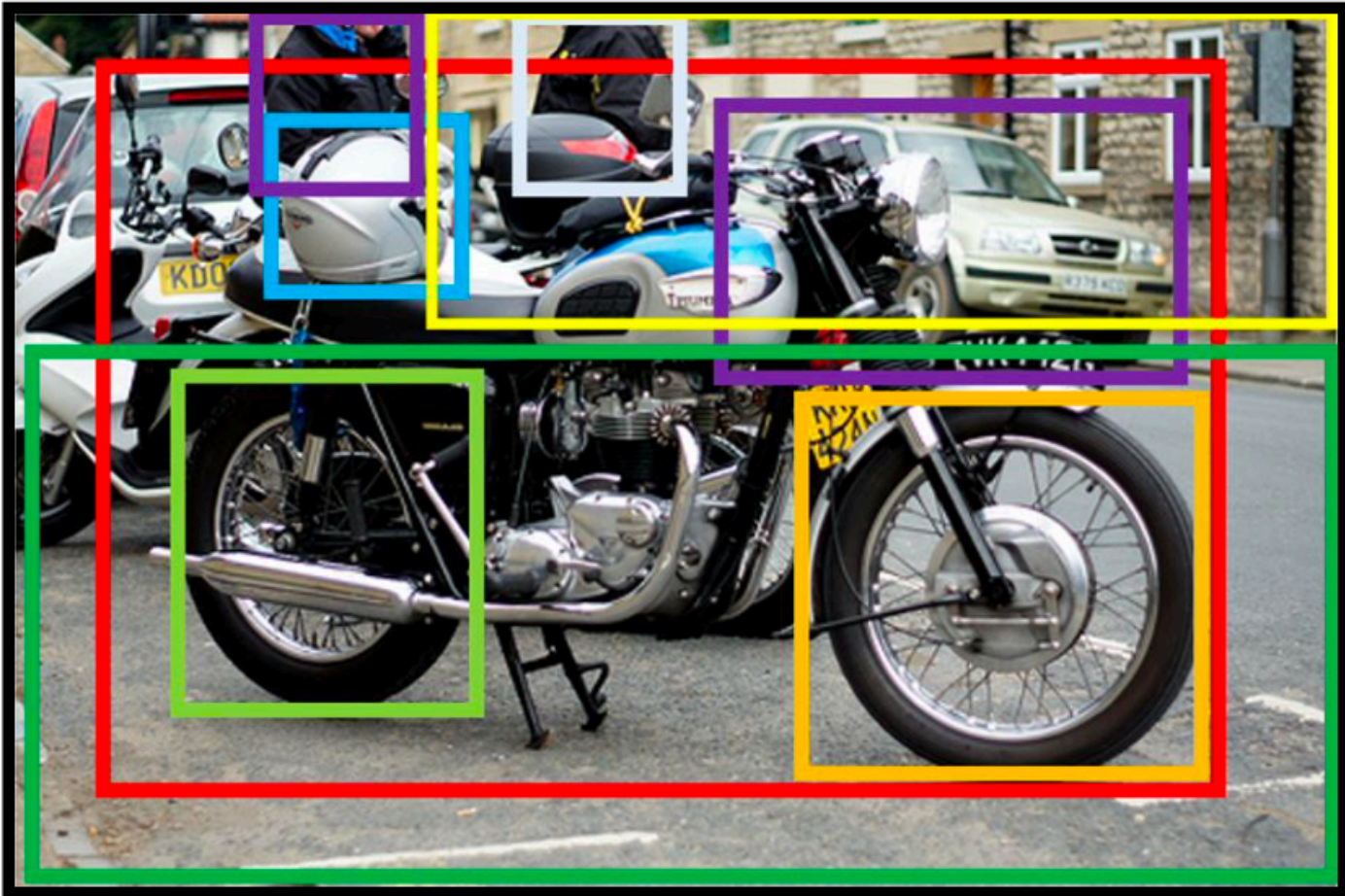


**Road network**

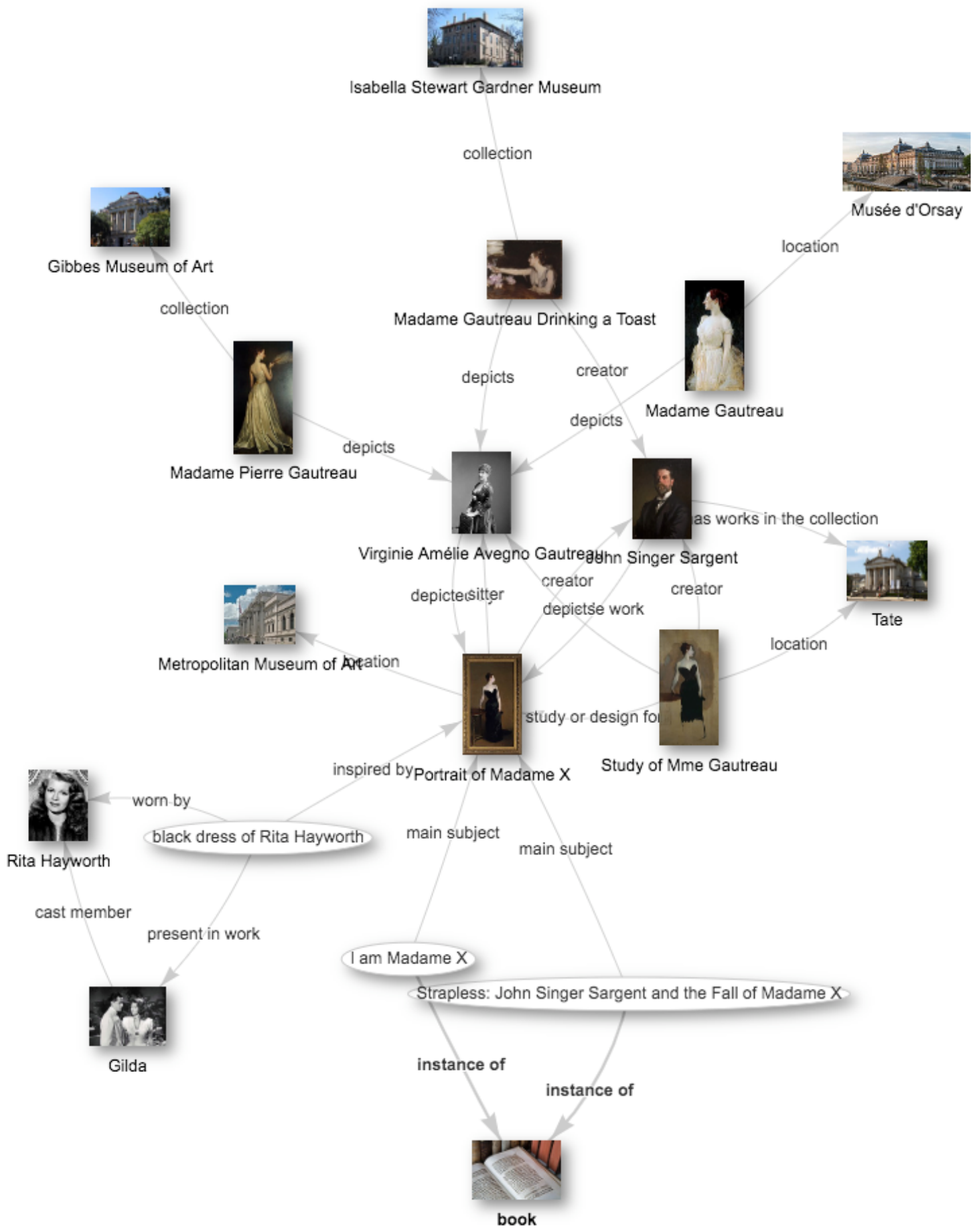


**Quadrilateral facial mesh**

# DATA AS GRAPHS

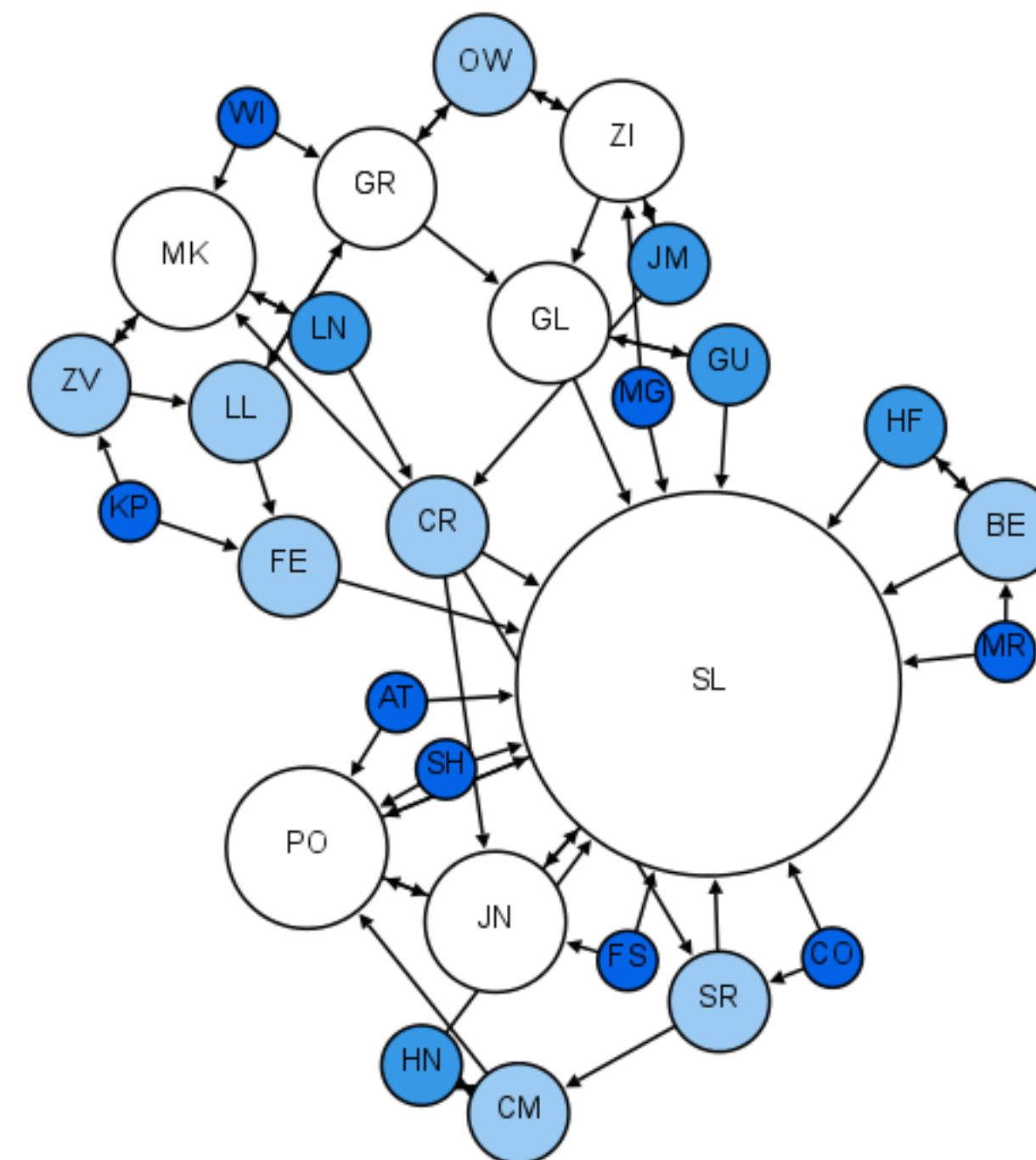
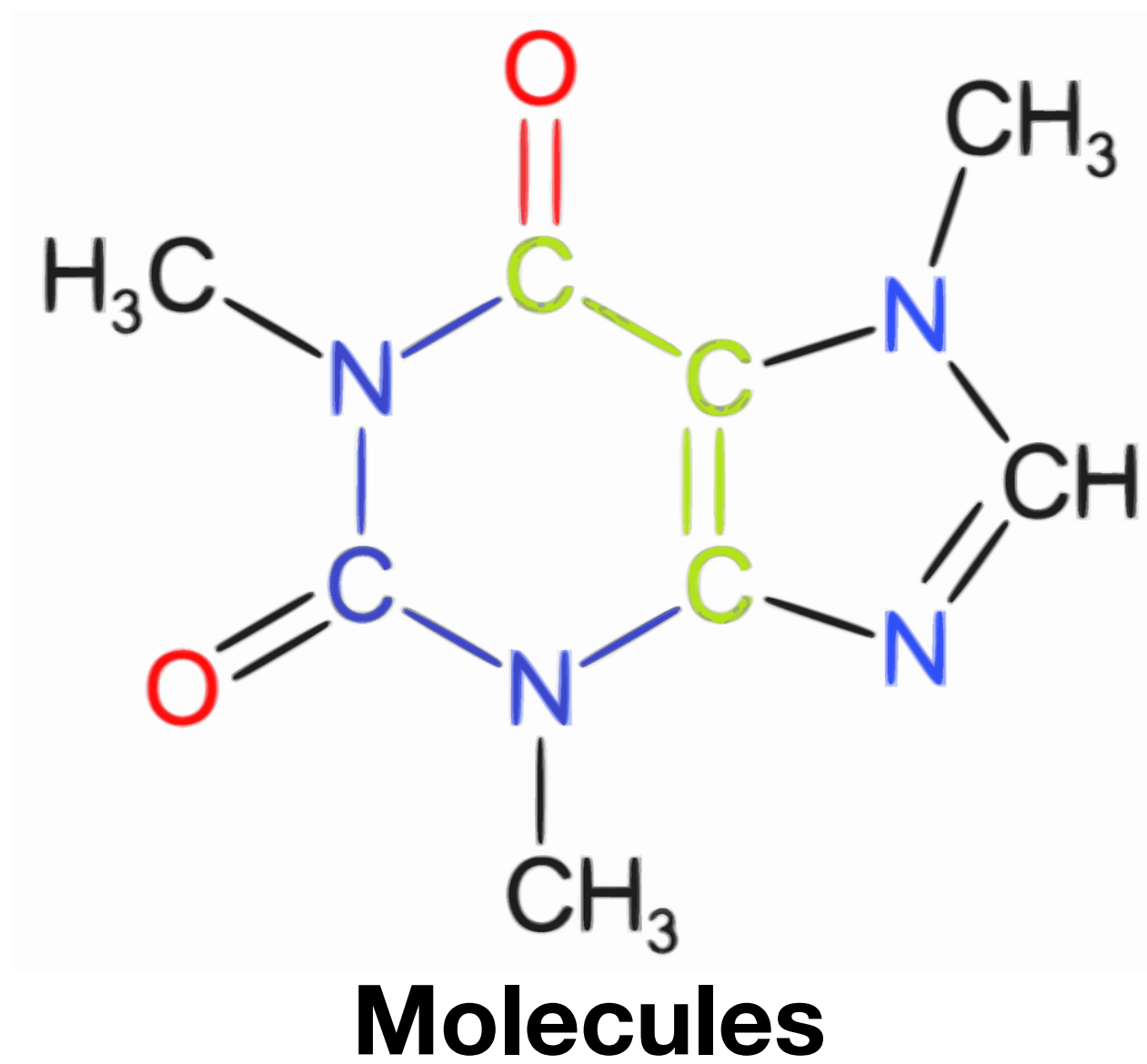


Scene graph

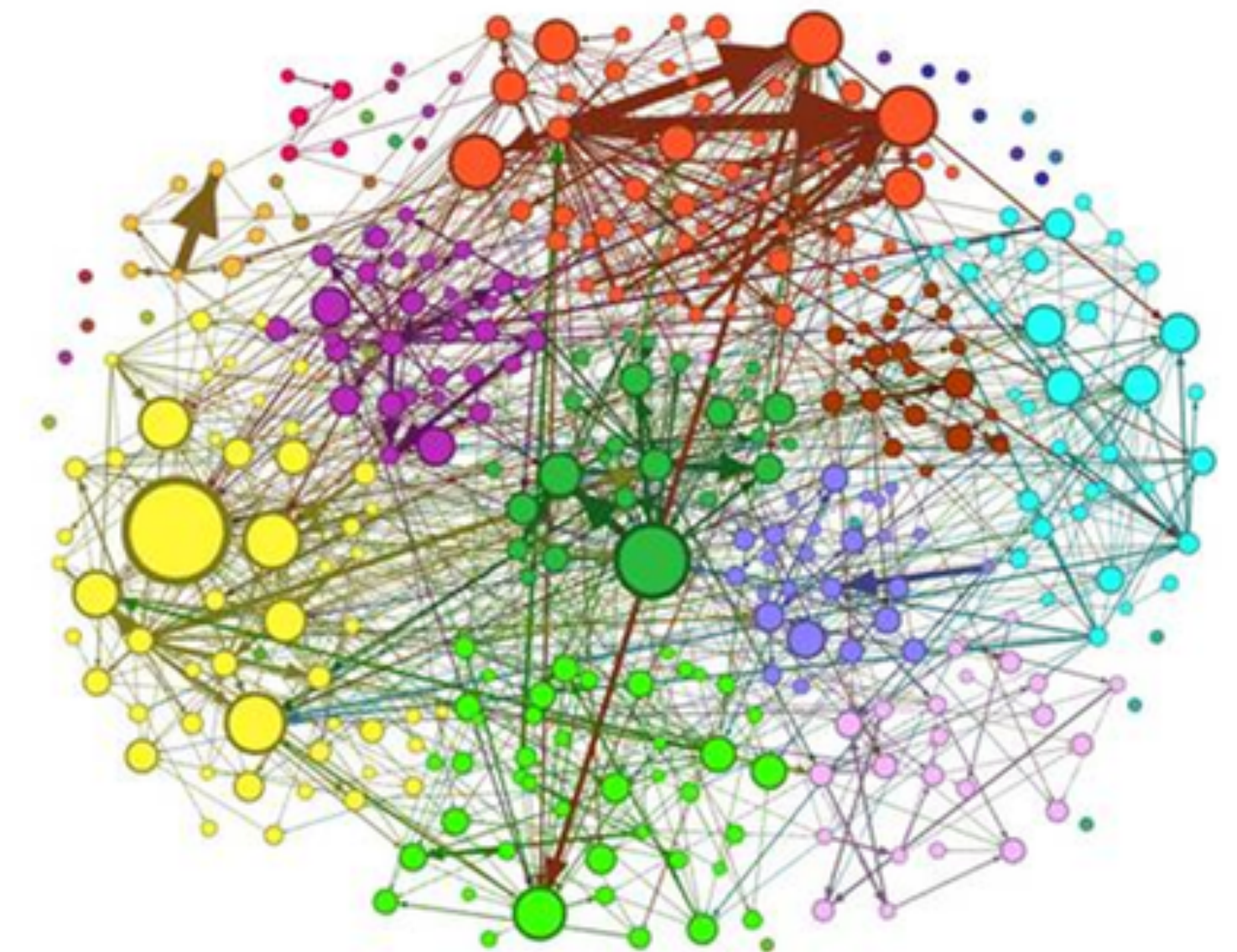


Knowledge graph

# DATA AS GRAPHS

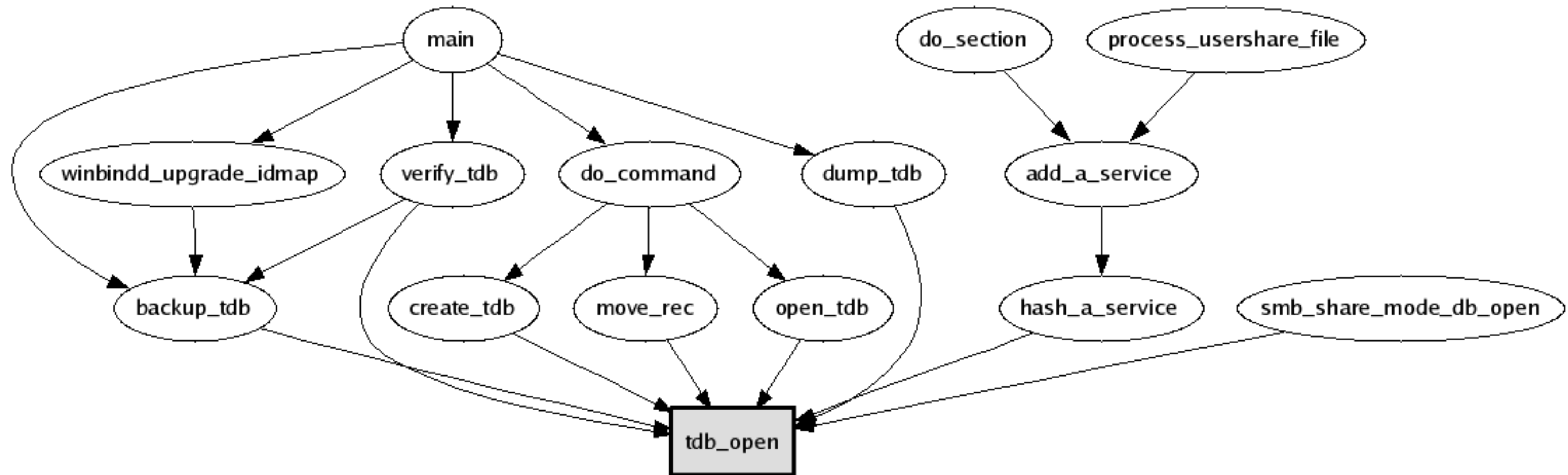


**Social graphs**



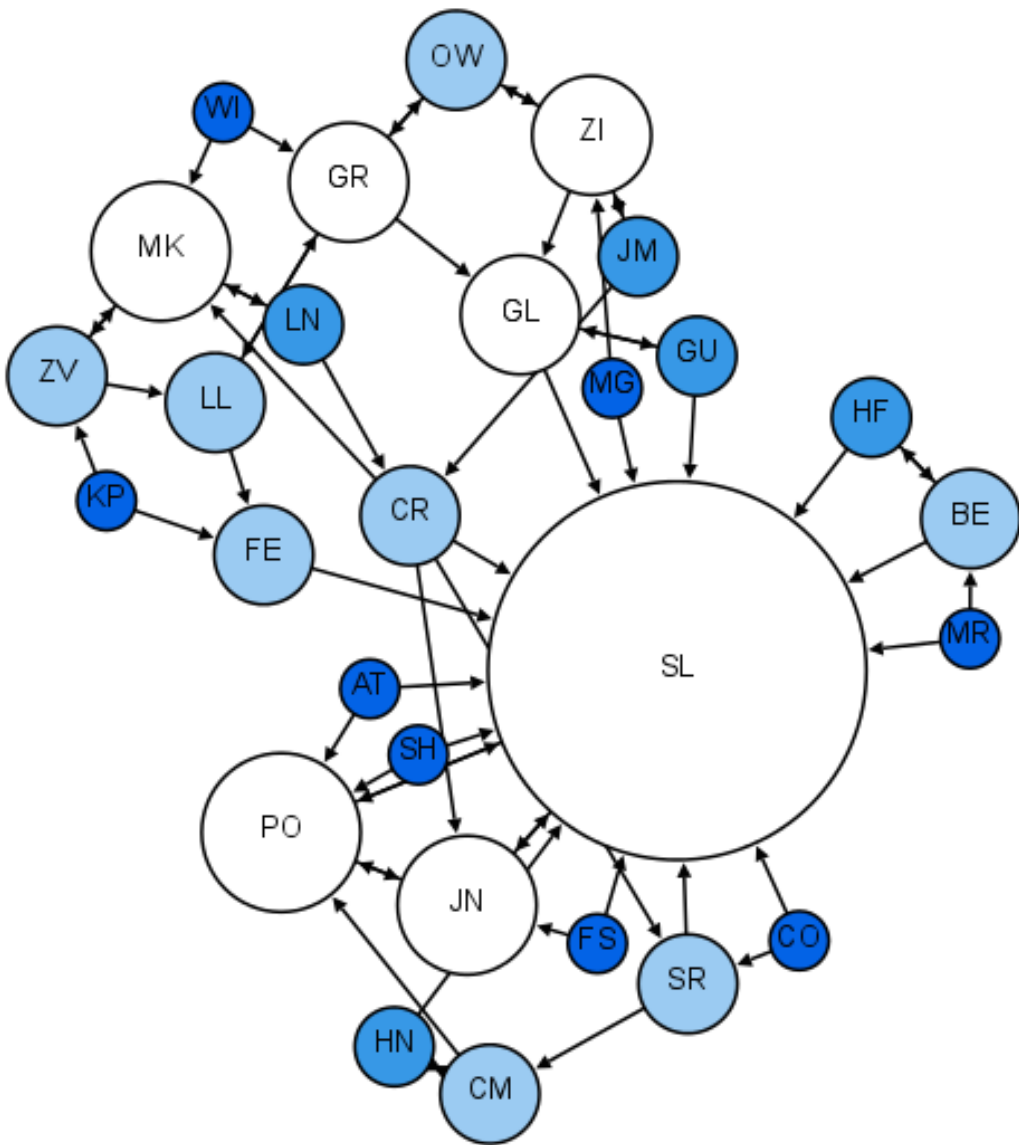
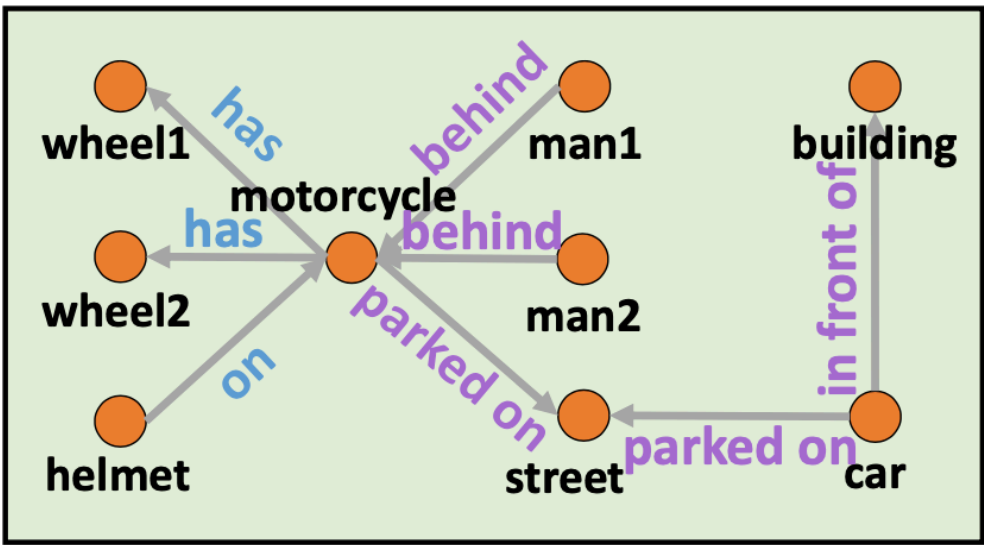
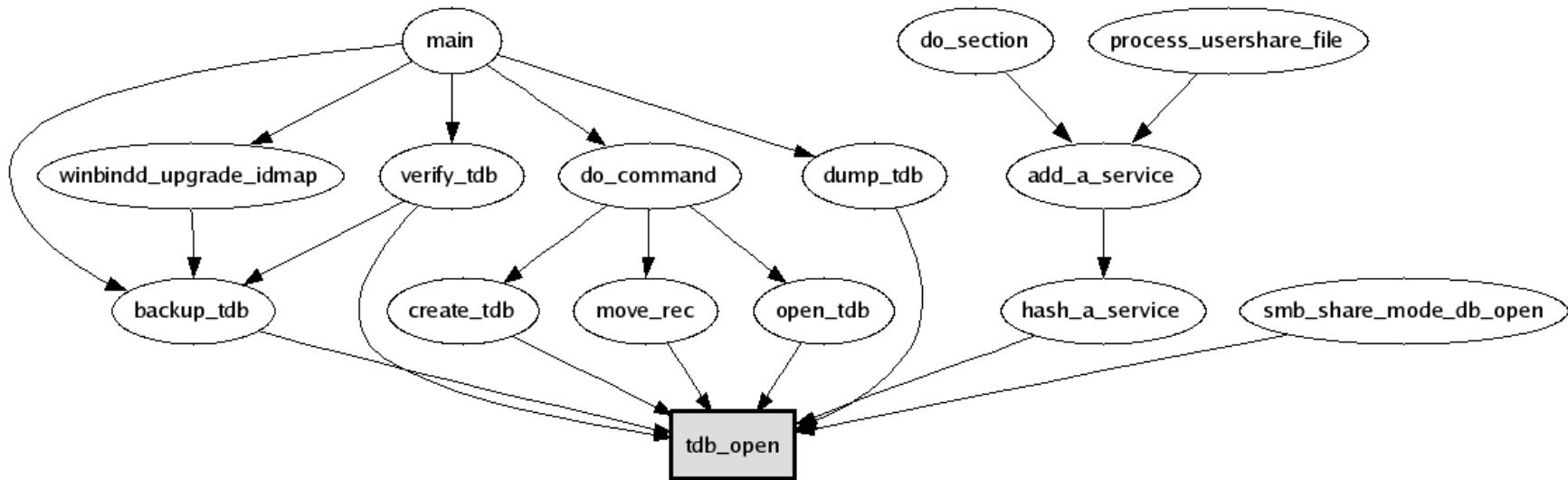
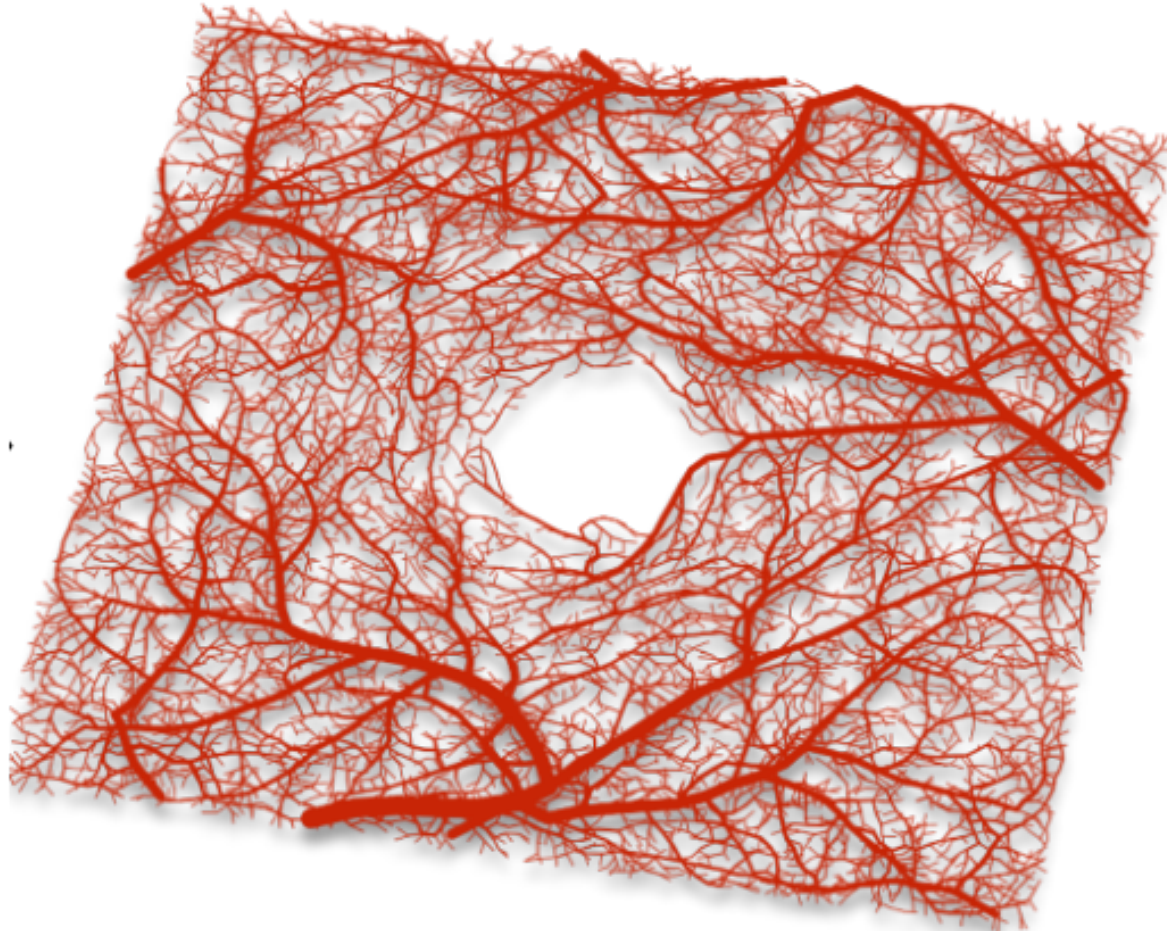
**Communication graphs**

# DATA AS GRAPHS



**Software code graph**

# STRUCTURAL VS. RELATIONAL GRAPHS



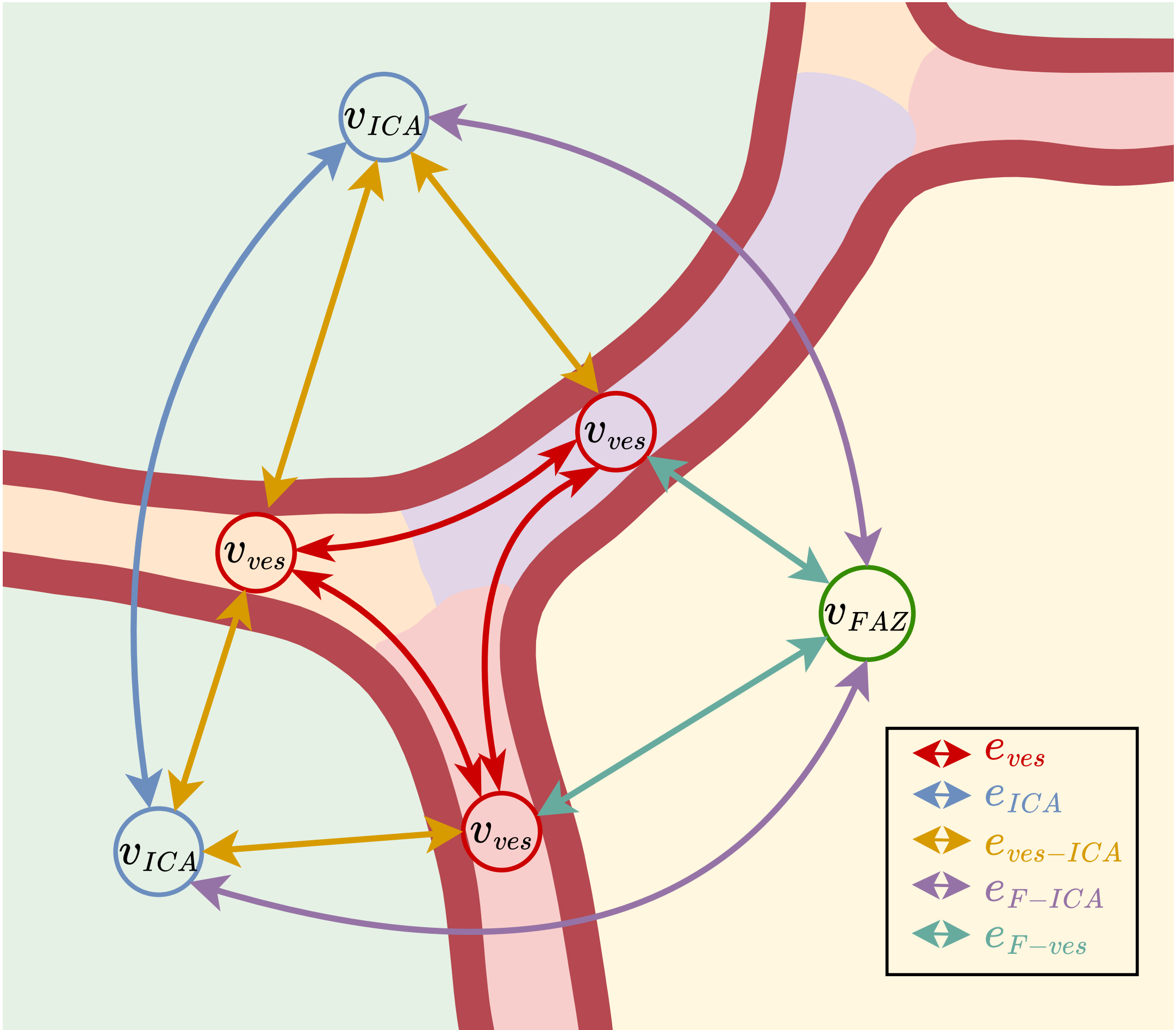
Structural

Relational

# DATA AS GRAPHS

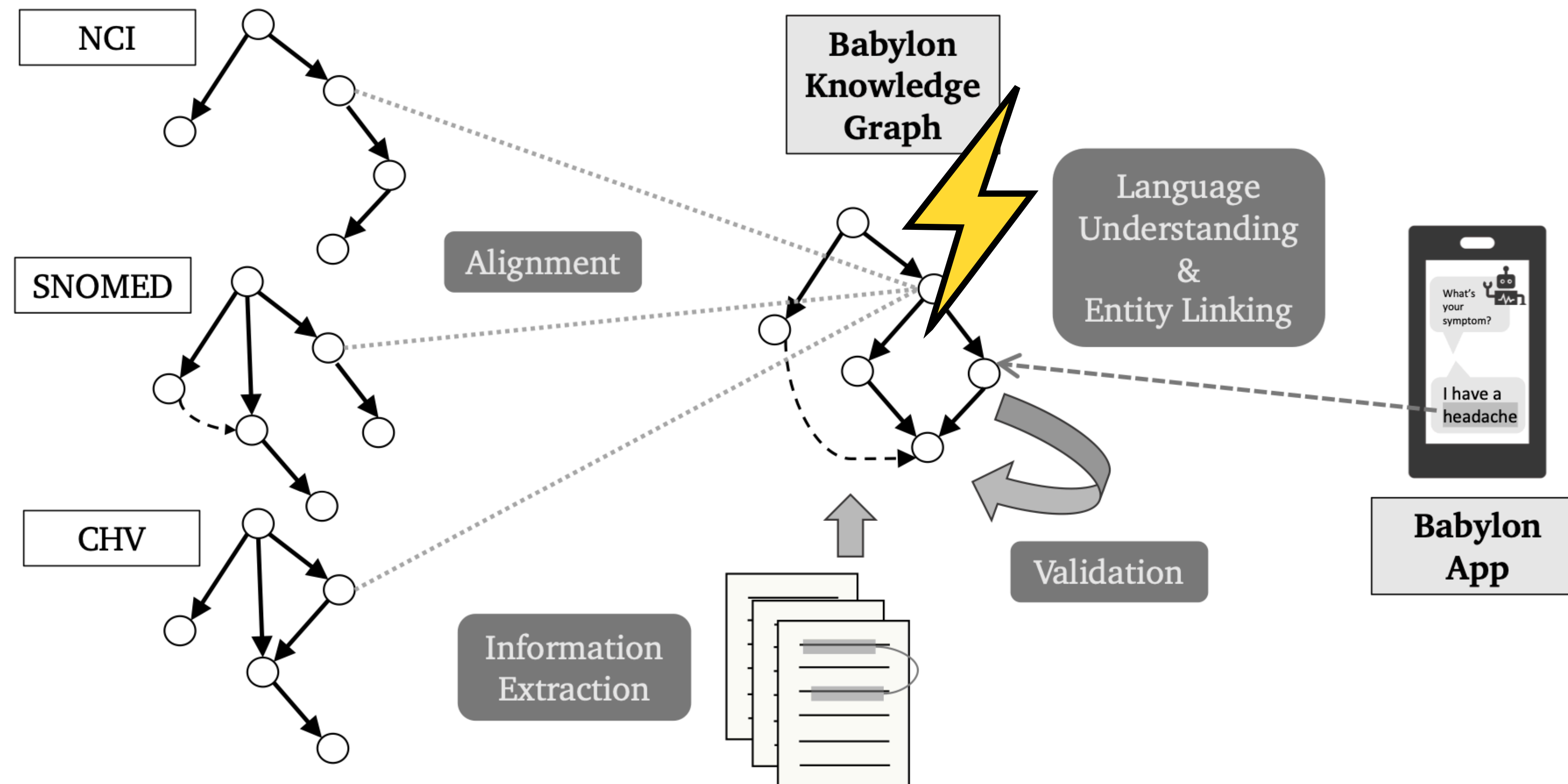
- Many data types are **naturally represented** as graphs
- Examples span **structural, relational and hybrid graphs**
  - Structural graphs focus on the overall graph layout and topology
  - Relational Graphs focus on the relationship between individual entities

# DATA AS GRAPHS



**Heterogeneous vascular graph**

# DATA AS GRAPHS



**Medical knowledge graph for chatbot backend**

# DATA AS GRAPHS

- Many data types are **naturally represented** as graphs
- Examples span **structural, relational and hybrid graphs**
  - **Structural graphs** focus on the overall graph layout and topology
  - **Relational Graphs** focus on the relationship between individual entities
- If and how to construct the graph often involves **explicit design choices**
  - In some cases, there is a unique, unambiguous representation
  - In other cases, there are multiple plausible representations
- Choice of graph will determine the nature of the questions you can study
- **Graph quality is crucial**

# REMINDER: WEEK 1

MDL - THE DATA ACQUISITION PROCESS

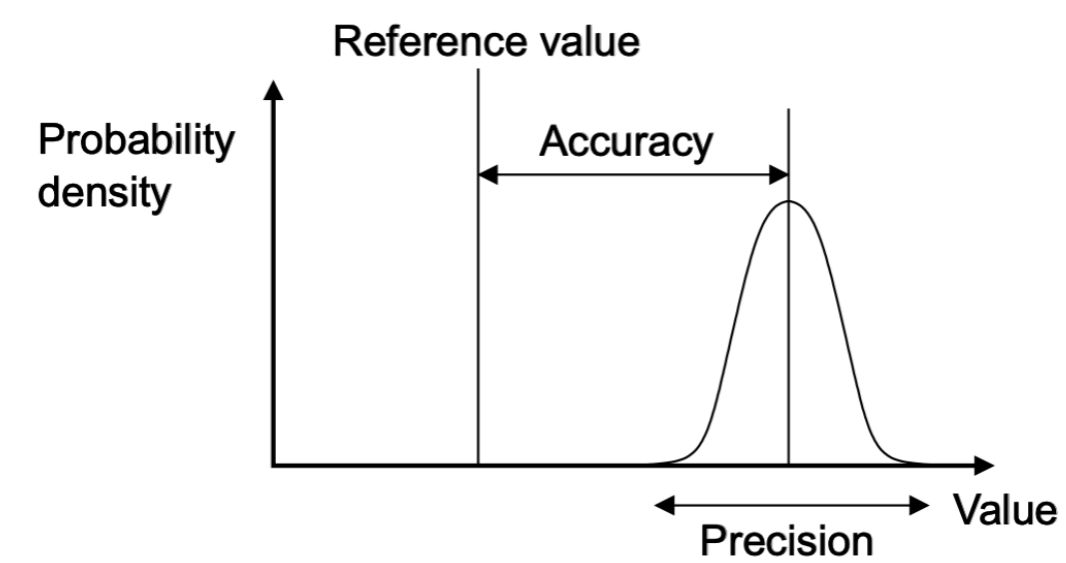
31 / 46 

## MEASUREMENT

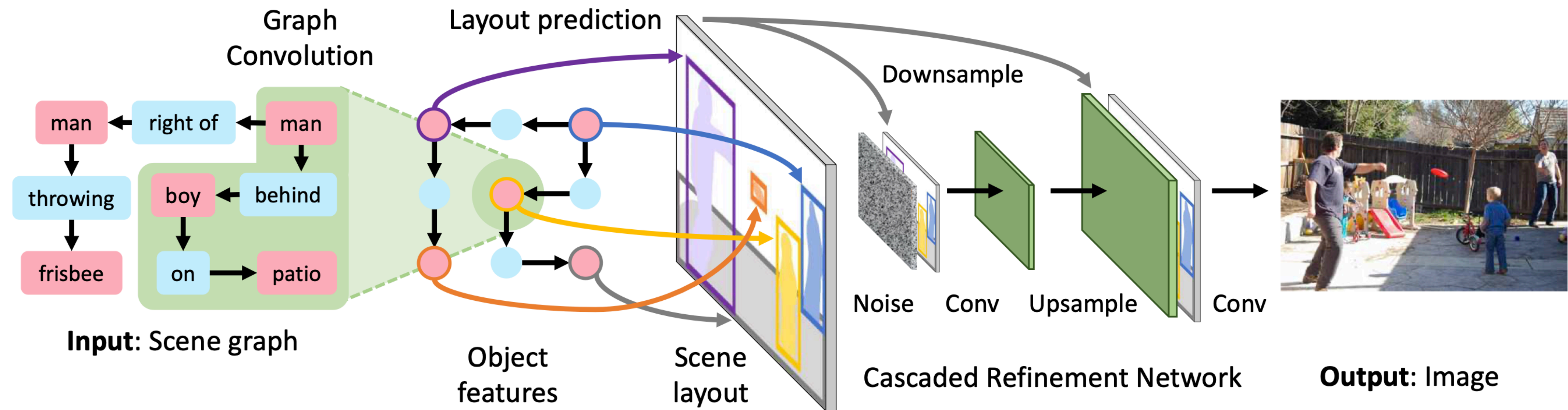
Measurement is the quantification of attributes of an object or event.

It should be:

- Accurate
- Precise
- Unbiased
- Reproducible



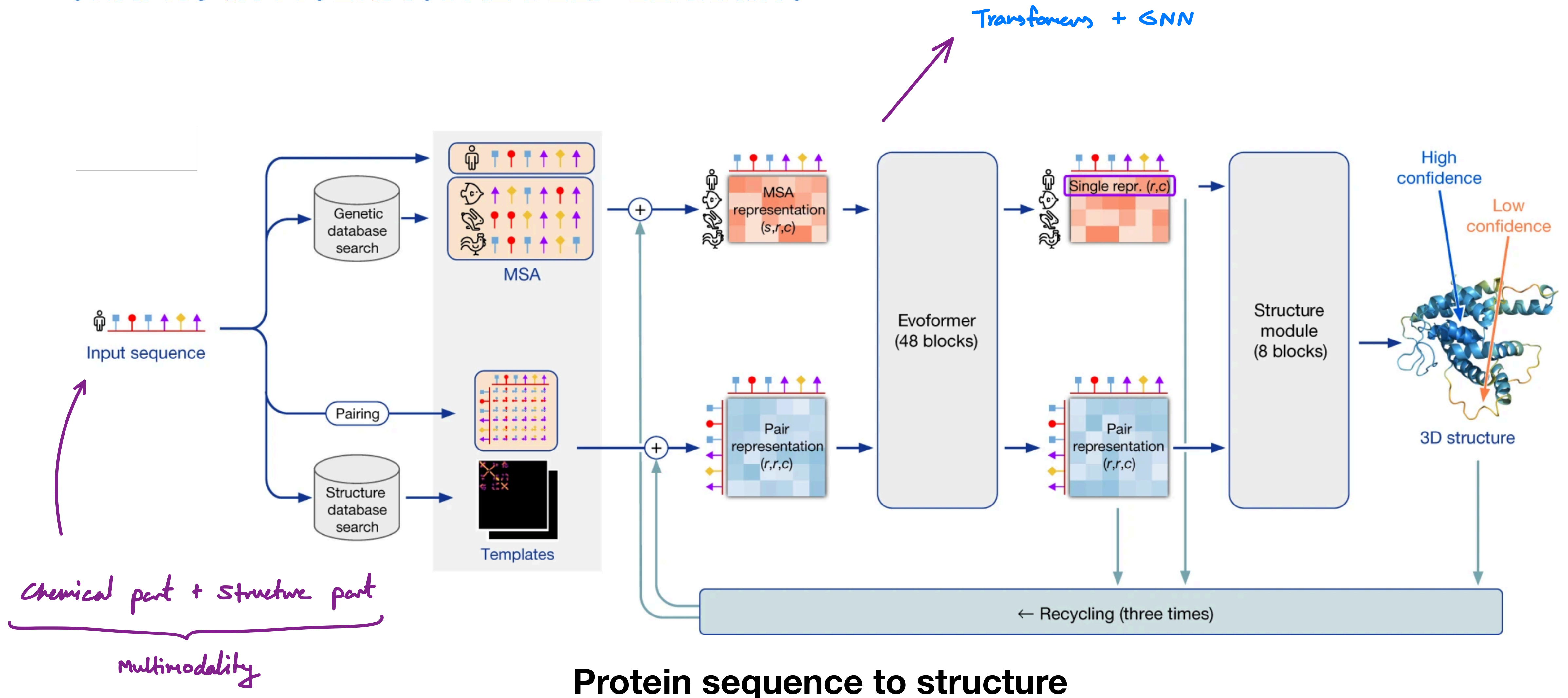
# GRAPHS IN MULTIMODAL DEEP LEARNING



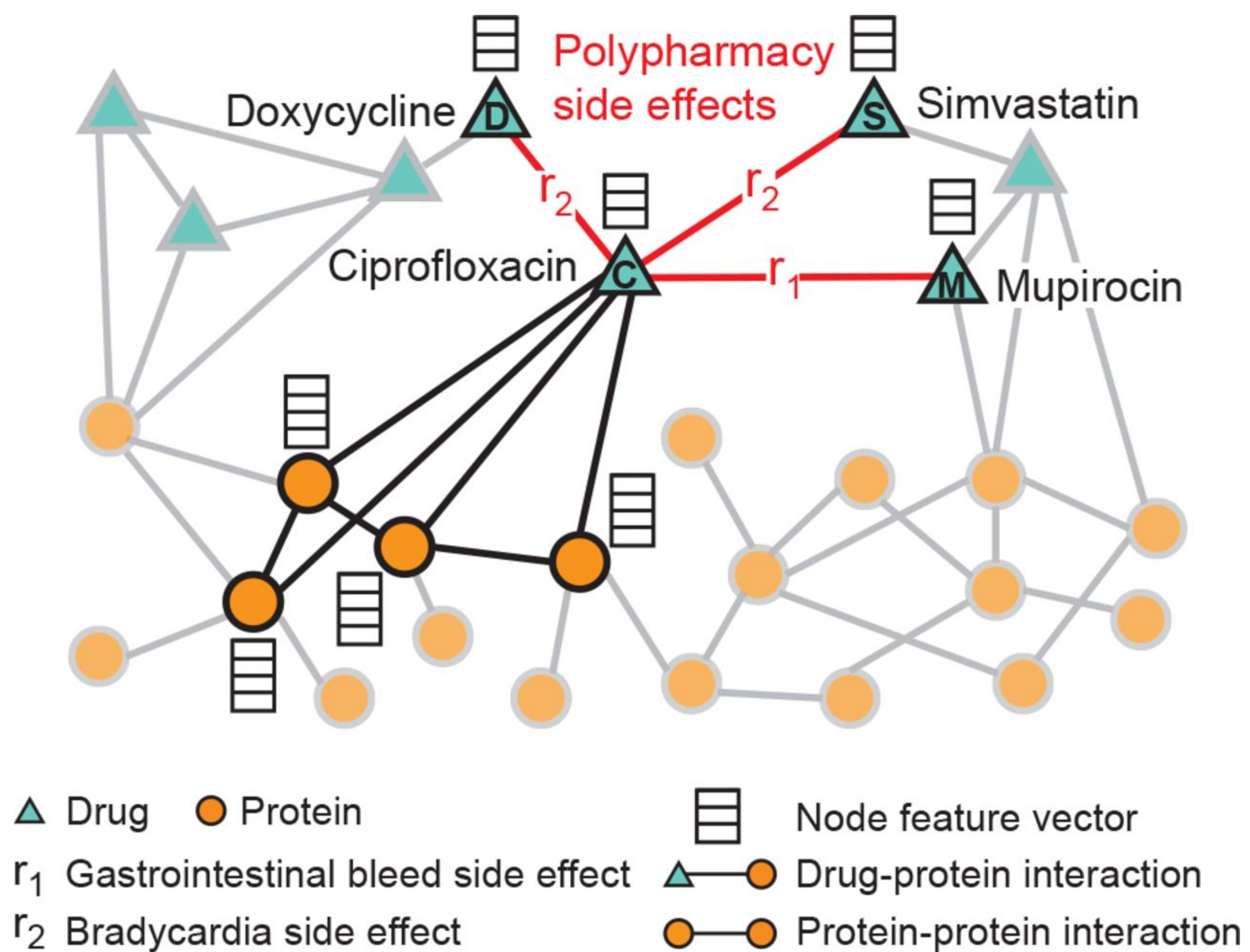
## Image generation from scene graphs

This idea wasn't implemented in 2018, but in 2025 with the advance computational resources.

# GRAPHS IN MULTIMODAL DEEP LEARNING



# GRAPHS IN MULTIMODAL DEEP LEARNING



Drug interaction graph

# GRAPHS IN MULTIMODAL DEEP LEARNING

- Graphs are commonly found in **multimodal deep learning**
  - Graph as **input** of a multimodal pipeline
  - Graph as **output** of a multimodal pipeline
  - Graph as **structure to relate** multimodal data embeddings

## THIS LECTURE

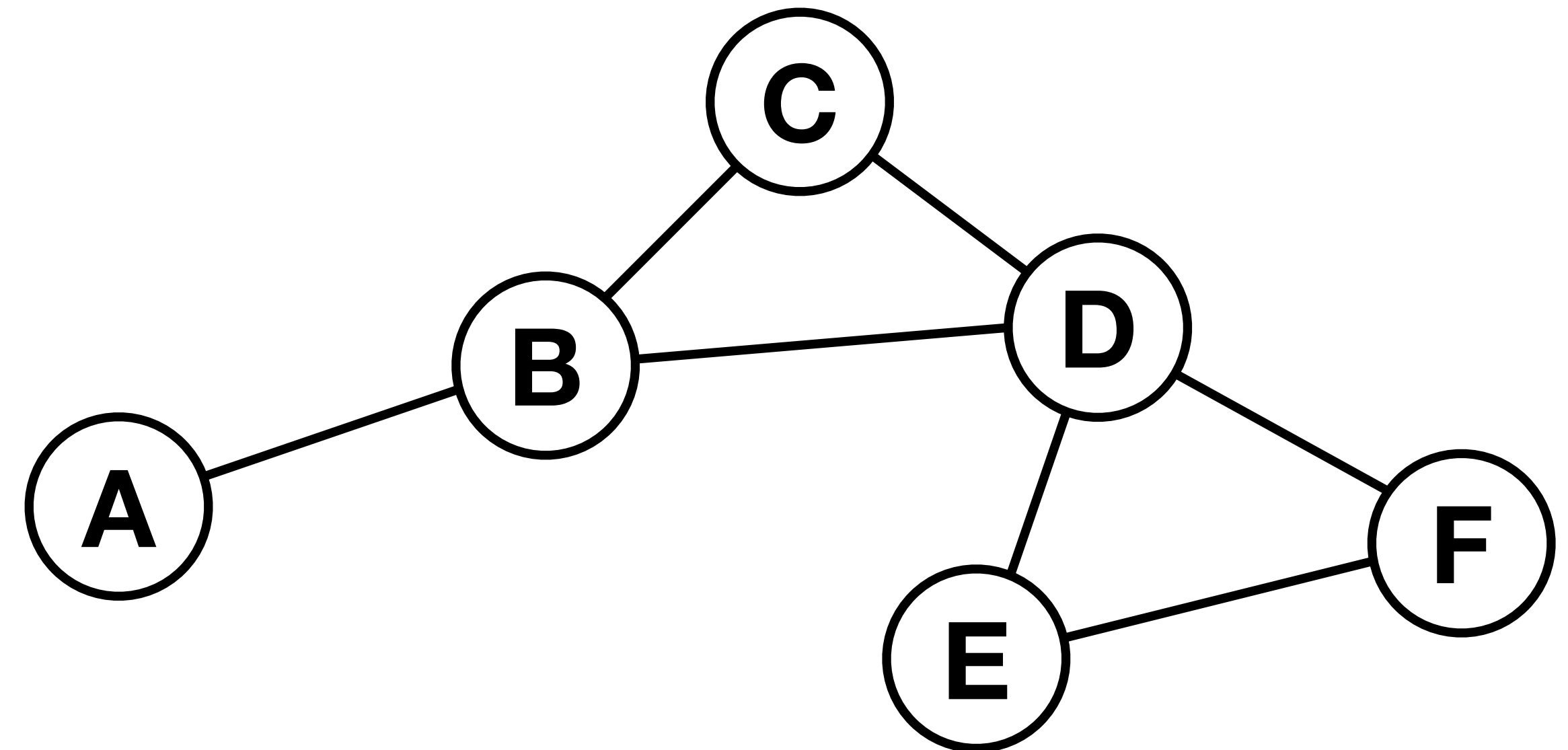
- Short introduction to graph deep learning
- Three strategies to make graphs multimodal
- Throughout: linking to previously introduced concepts of embeddings, encoders, and equivariance

# GRAPH AS LIST OF EDGES

A graph  $G = (V, E)$  is defined as a set of nodes  $v_i \in V$  connected by edges  $(v_i, v_j) \in E$

$$V = \{A, B, C, D, E, F\}$$

$$E = \{(A, B), (B, C), (B, D), (C, D), (D, E), (D, F), (E, F)\}$$

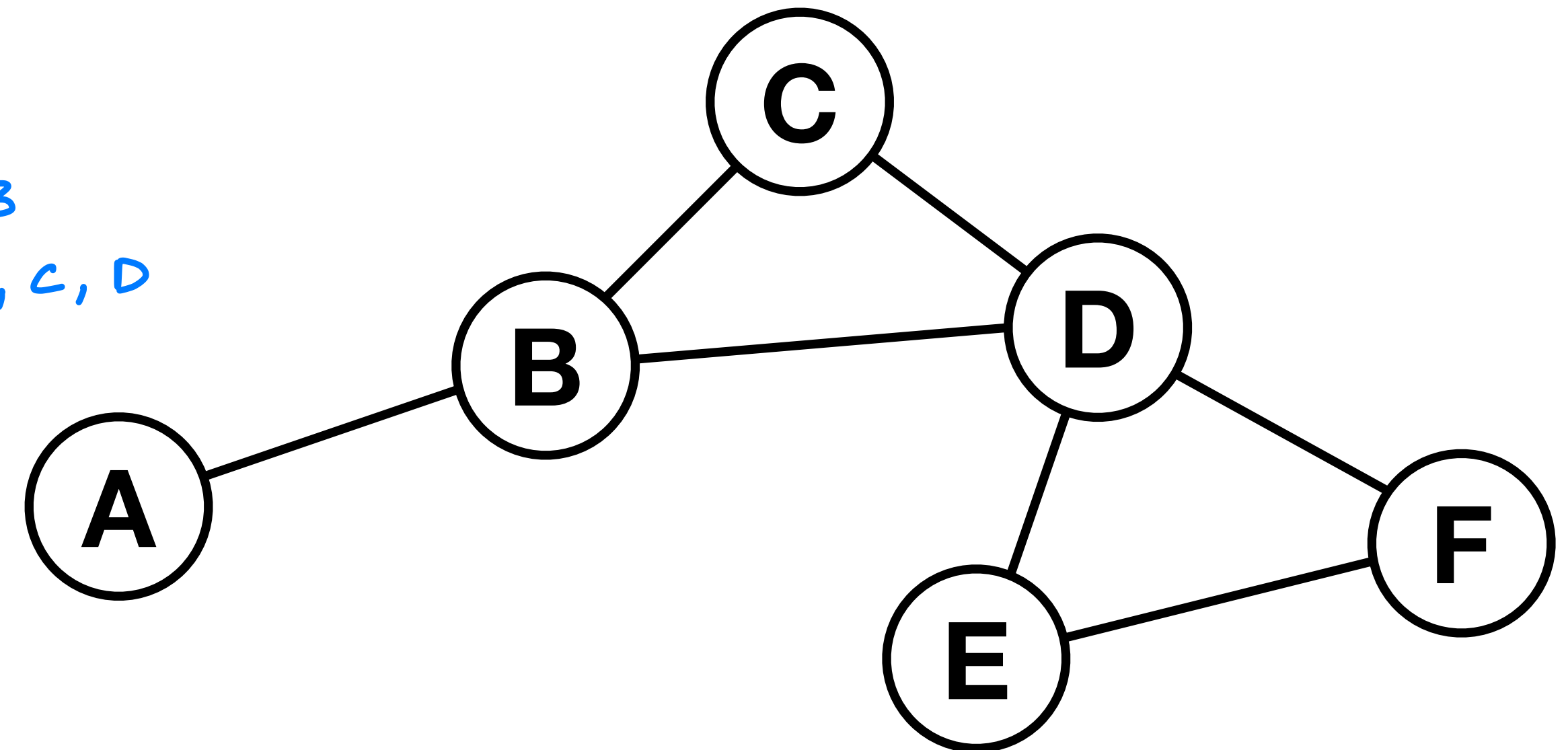


# GRAPH AS **ADJACENCY MATRIX** $\longrightarrow$ look easy but high memory consumption (email graph) Plus, super sparse.

Adjacency matrix  $A$  can be used as alternative representation

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Node B  
adjacent to A, C, D

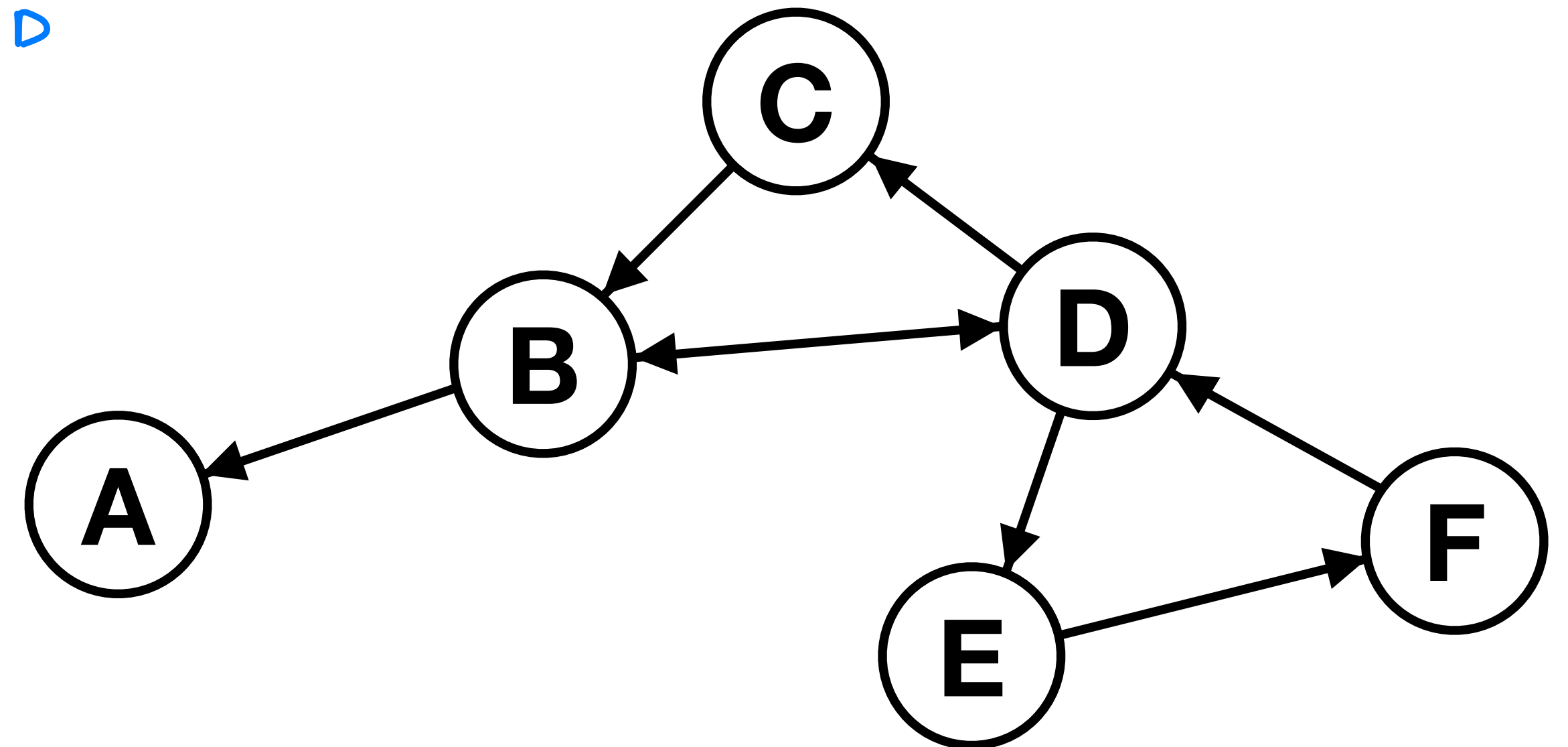


# DIRECTED GRAPH

Edges can be undirected:  $(v_i, v_j) = (v_j, v_i)$  or directed,  $(v_i, v_j) \neq (v_j, v_i)$

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

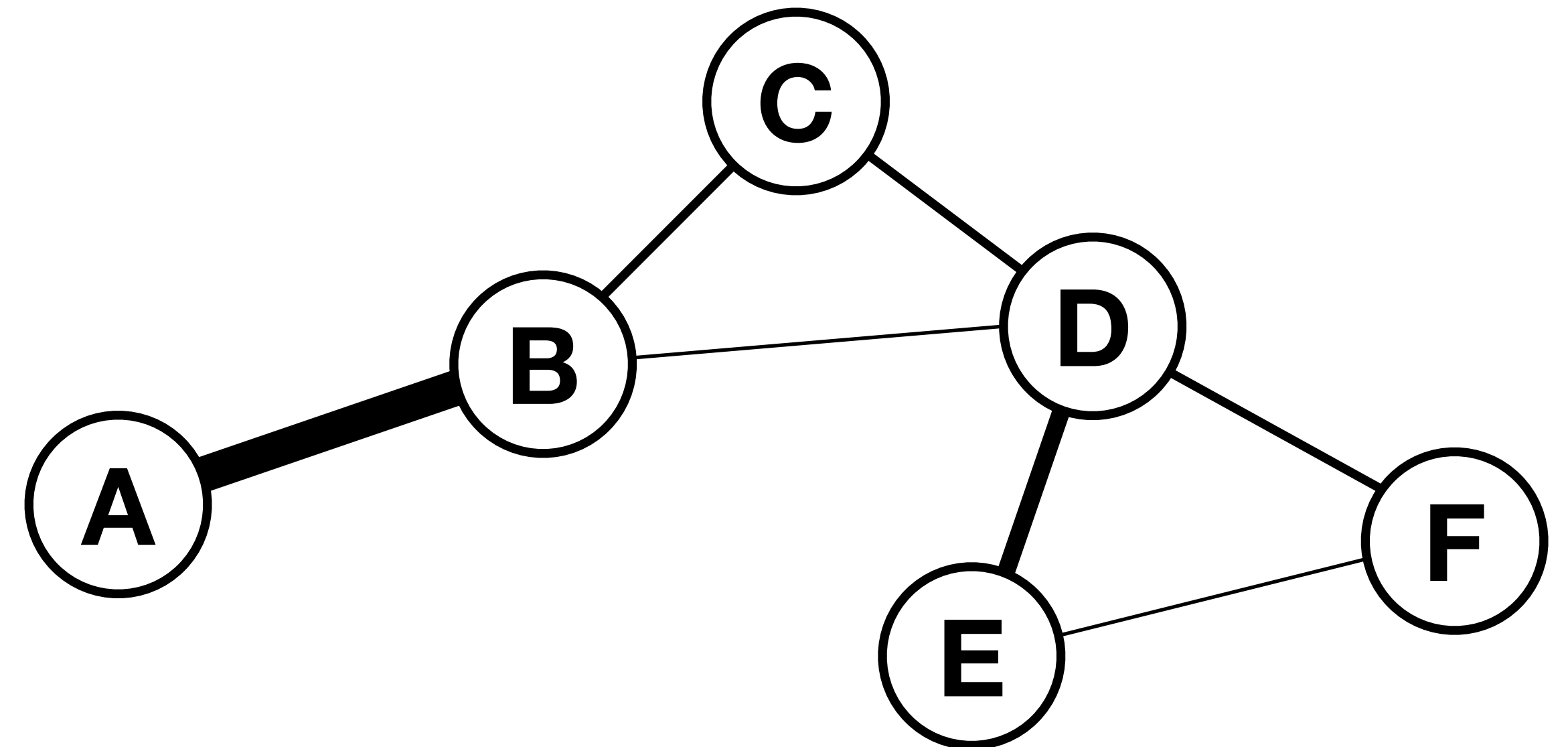
B to A and D



## EDGE FEATURES

Edges can also be weighted or imbued with more complex embeddings

$$A = \begin{bmatrix} 0 & 2 & 0 & 0 & 0 & 0 \\ 2 & 0 & 1 & 0.5 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0.5 & 1 & 0 & 1.5 & 1 \\ 0 & 0 & 0 & 1.5 & 0 & 0.5 \\ 0 & 0 & 0 & 1 & 0.5 & 0 \end{bmatrix}$$

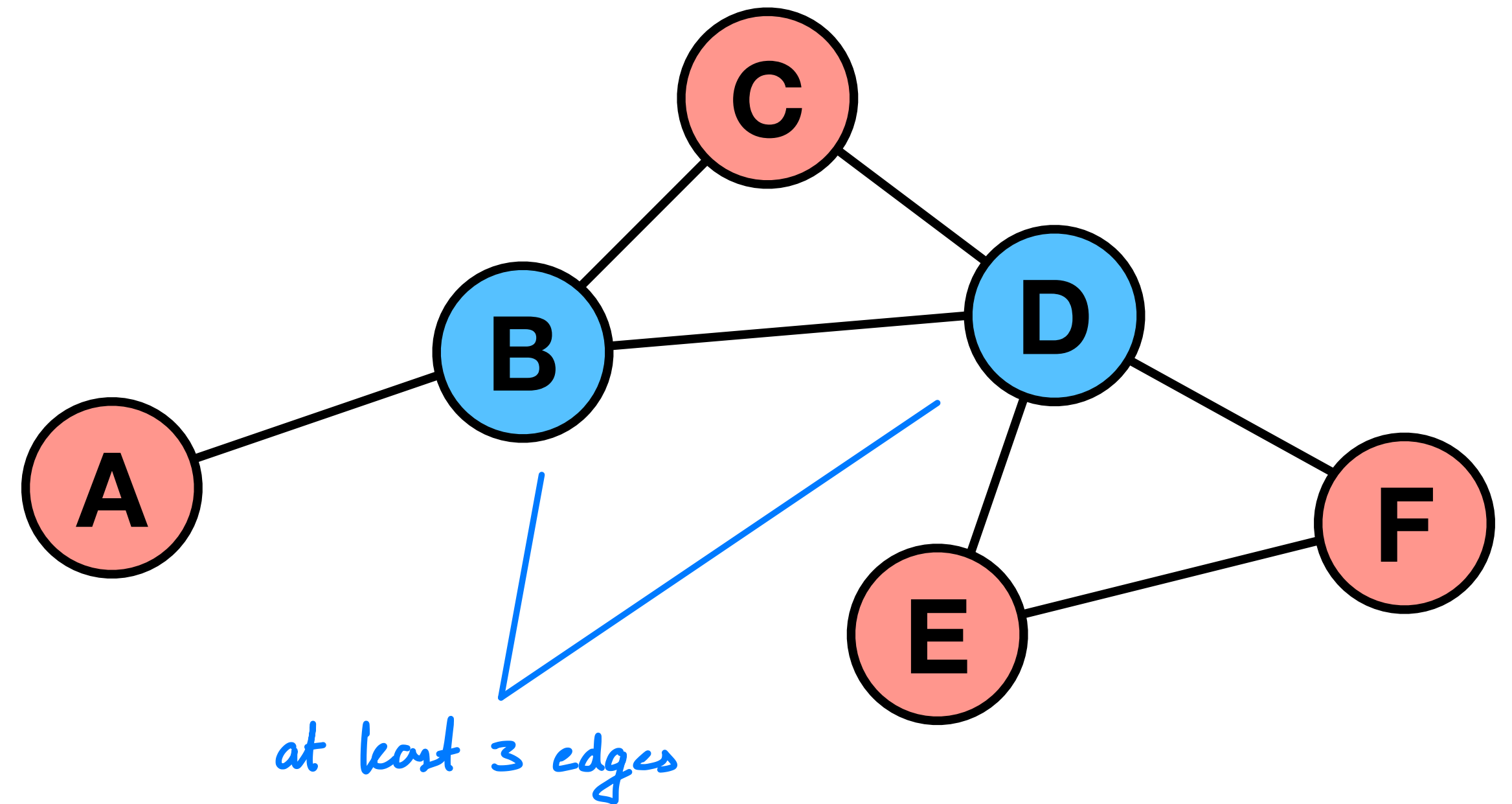


# NODE FEATURES

Each node is imbued with a  $d$ -dimensional feature vector

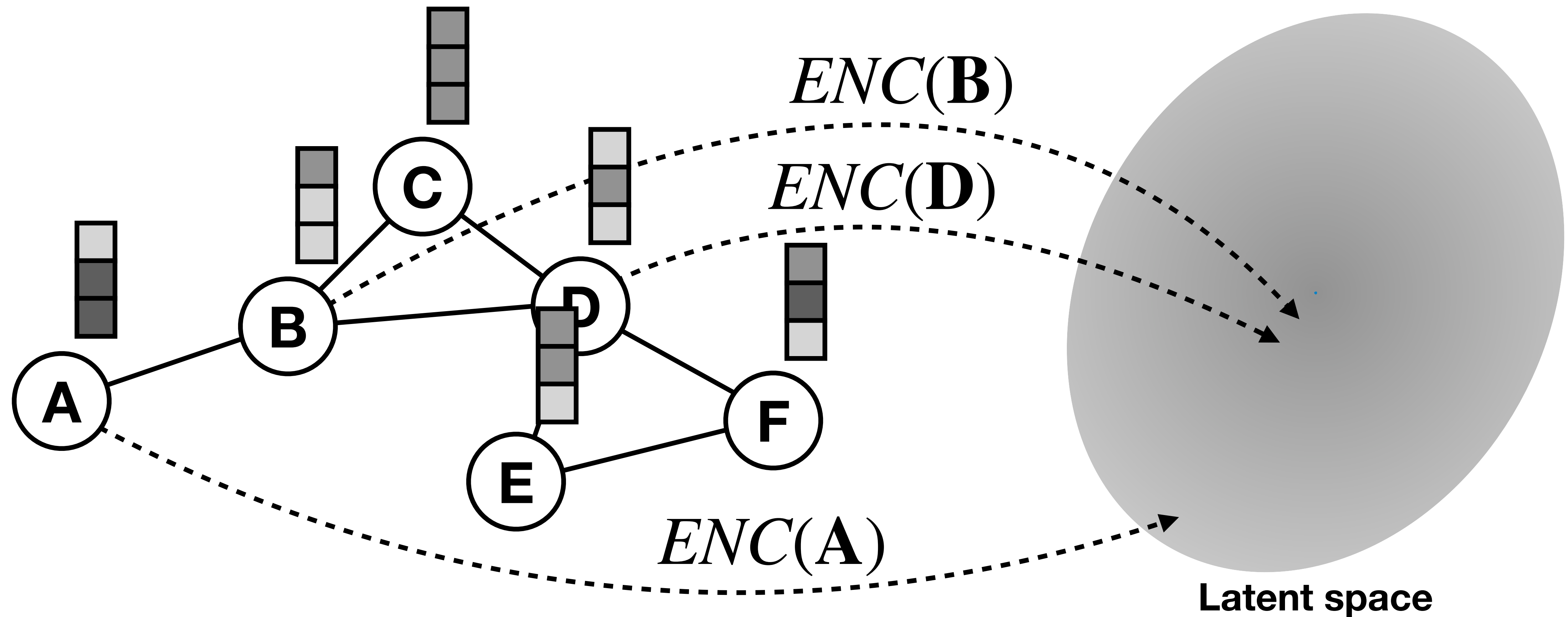
$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$$F = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}$$



# NODE FEATURES

Mapping of nodes to latent space:  $\text{sim}(\mathbf{v}_i, \mathbf{v}_j) \sim \mathbf{z}_{\mathbf{v}_i}^T \mathbf{z}_{\mathbf{v}_j}$

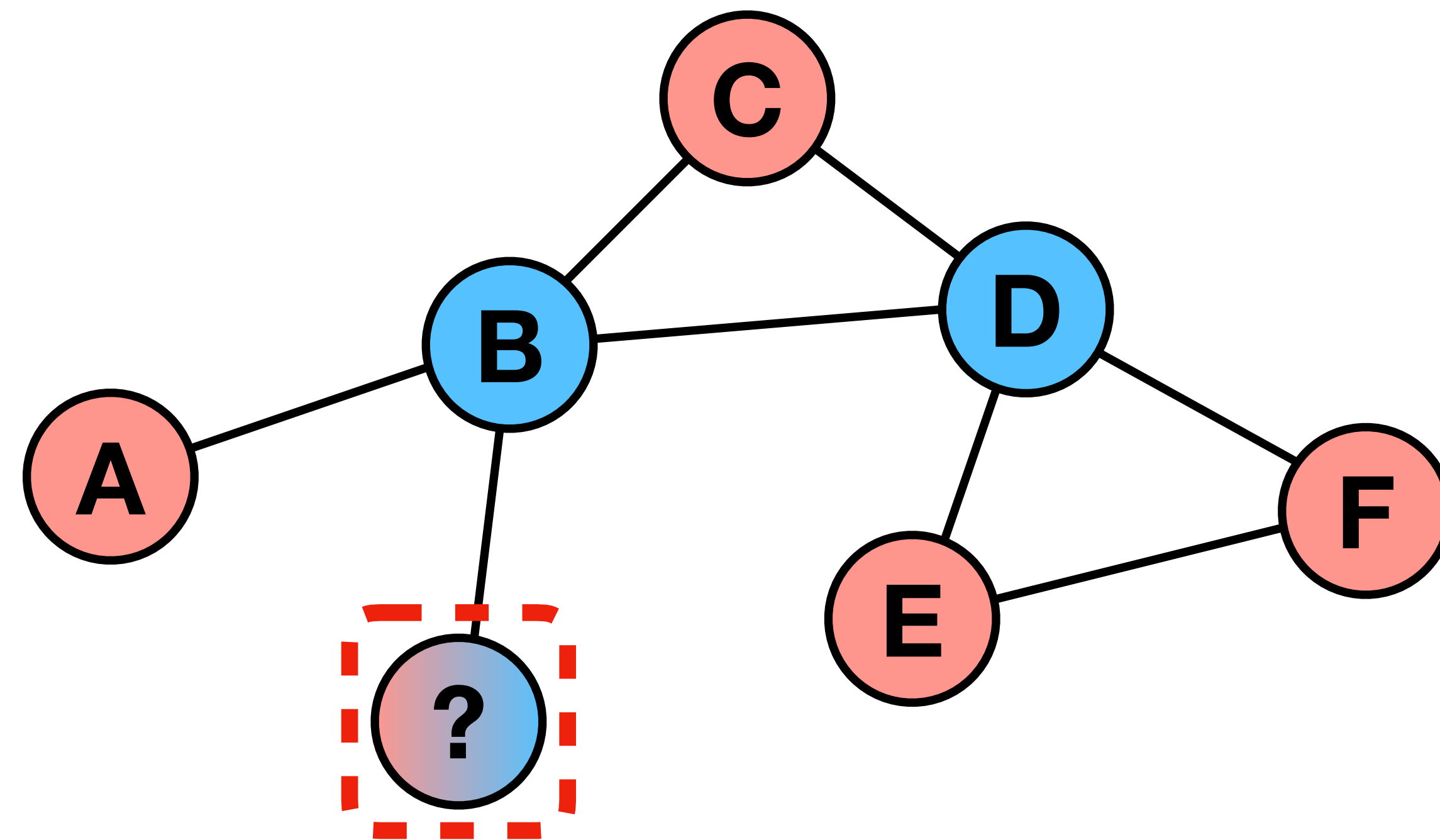


# CLASSICAL NODE EMBEDDING STRATEGIES

- Feature engineering based on prior knowledge or graph structure (number of neighbors, node centrality, ...)
- Kernel-based methods with pre-defined graph kernels
- Spectral methods based on Laplacian eigenmaps
- Random walk based strategies (node2vec, ...)

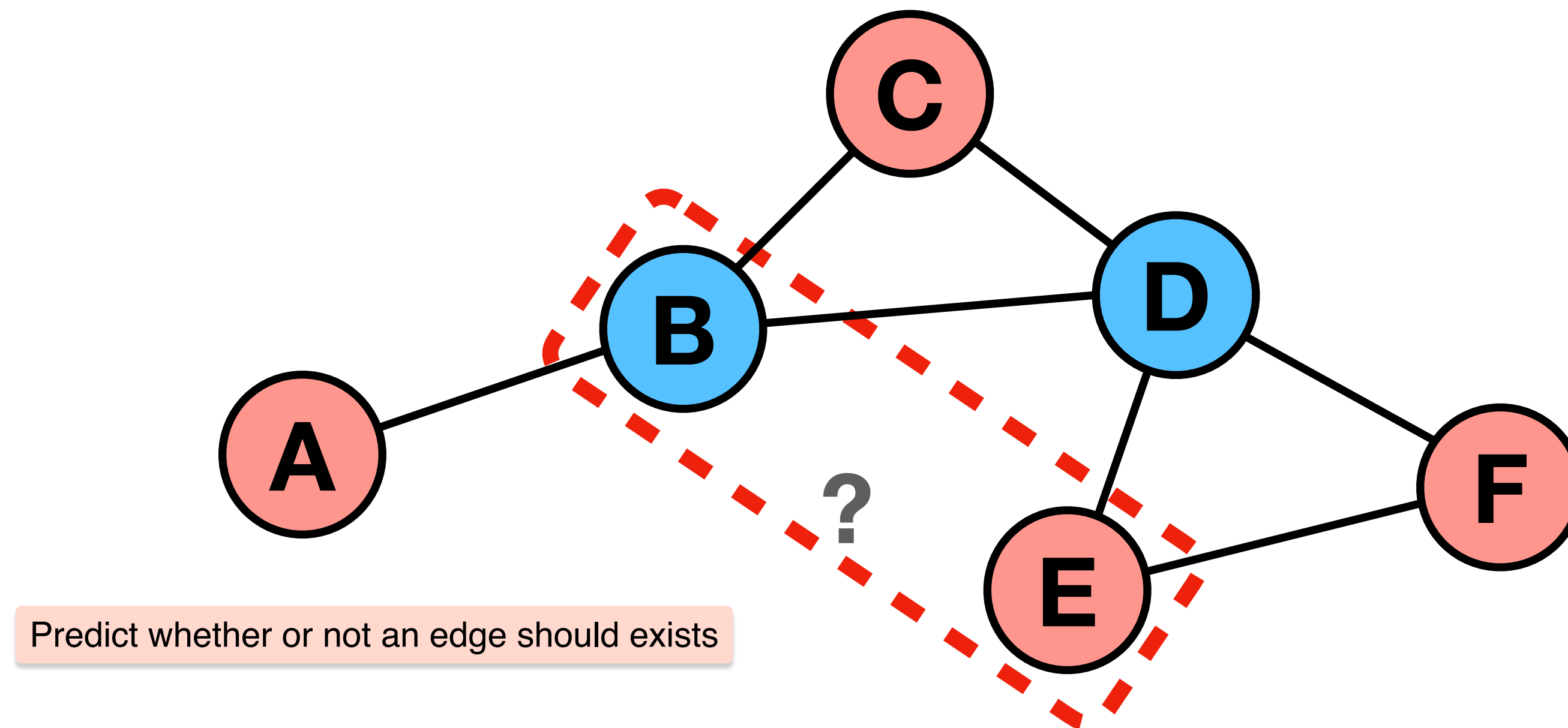
# NODE-LEVEL PREDICTION

Evaluation of node features to predict node class



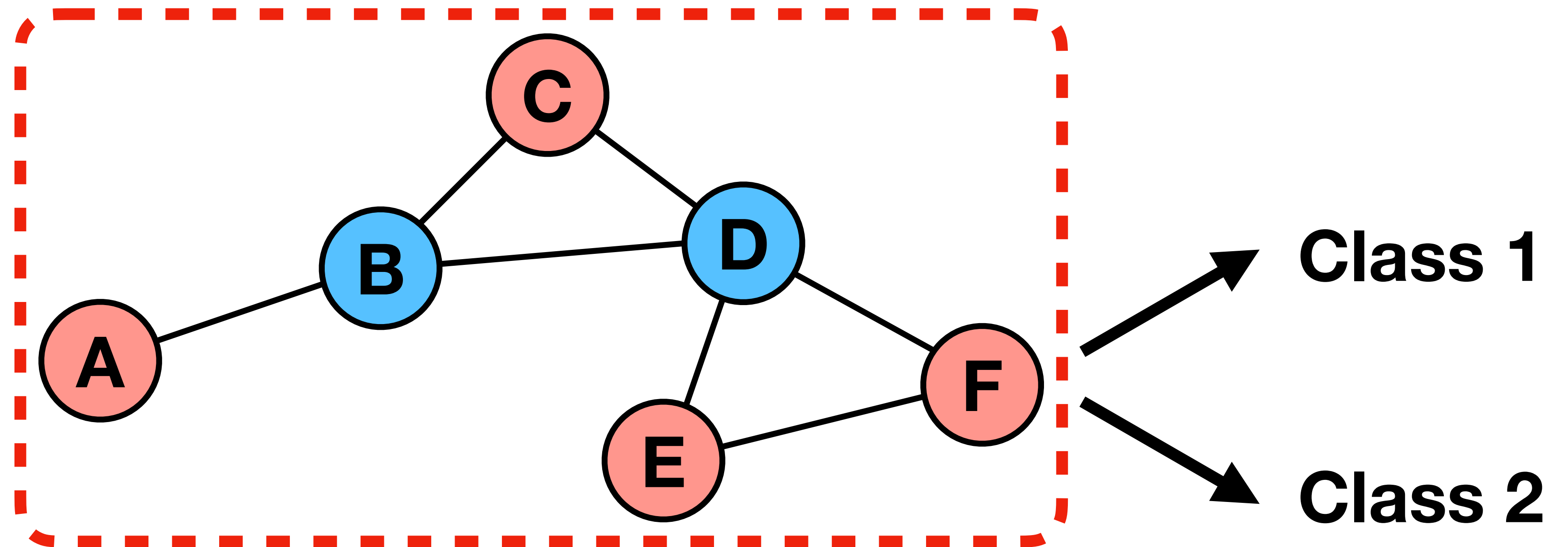
## EDGE-LEVEL PREDICTION

Evaluation of edge features (e.g. features of neighboring nodes) to predict existence or weight of potential edge



## GRAPH-LEVEL PREDICTION

Evaluation of graph features (e.g. by aggregating all node features) to make graph-level prediction



# NODE FEATURES

Manifold space ←

- Generation of **node features by mapping of nodes to a latent space**
  - Similar nodes should obtain similar embeddings,
  - Dissimilar nodes should obtain distant embeddings
- Node features allow us to solve diverse downstream tasks
- Classical node embedding strategies may lack expressiveness

**Can we use deep learning to  
derive useful node embeddings?**

# REMINDER: WEEK 2

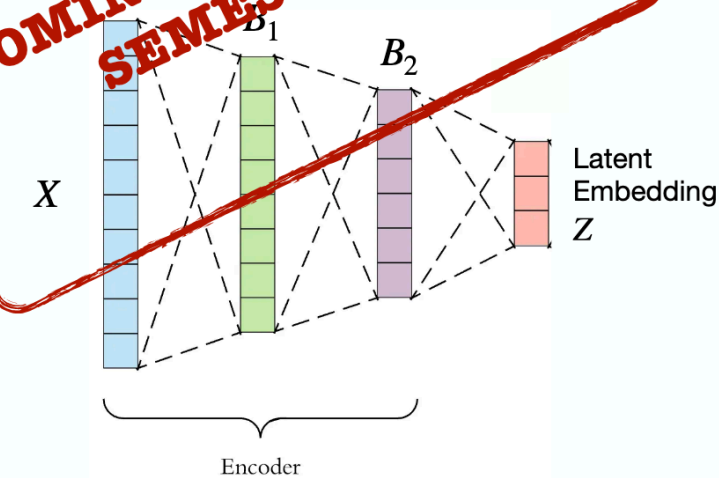
## LEARNING NON-LINEAR EMBEDDINGS

💡 To capture **even more** complex semantic relationships, learning **non-linear** embeddings is required.

### Neural Network Encoders

Modern encoders learn embeddings through hierarchical, structured, and compositional non-linear transformations.

- **Note:** single-layer MLP without activation is simply a linear mapping ( $B = AX + \beta$ ).
- Each intermediate hidden representation in an encoder is an embedding.

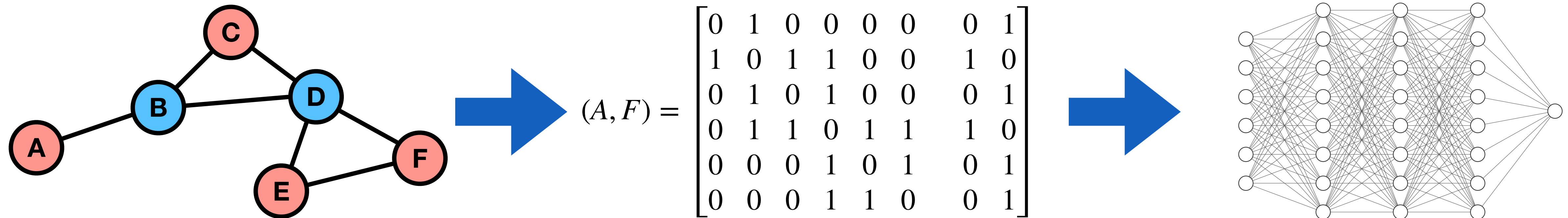


[7] Arden Dertat. <https://medium.com/towards-data-science/applied-deep-learning-part-3-autoencoders-1c083af4d798> (accessed January 2025).

# A NAIVE APPROACH

Concatenation

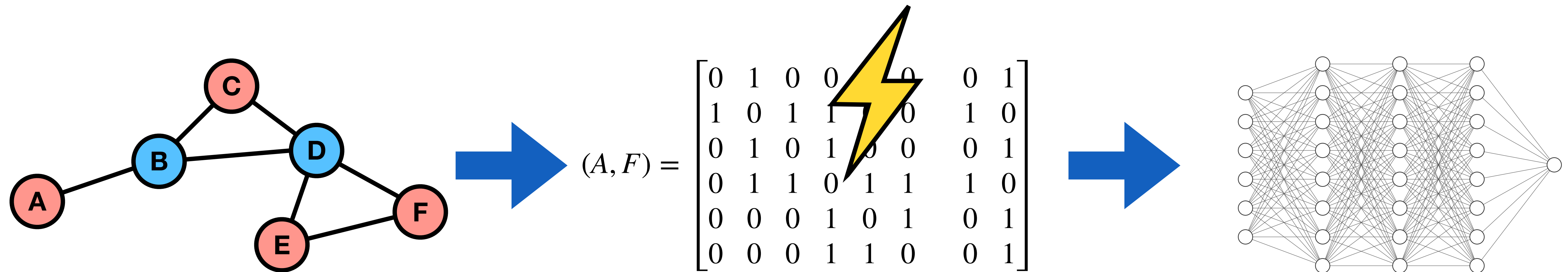
Represent graph as adjacency and feature matrix and process with a multi-layer perceptron



# A NAIVE APPROACH

Problem #1:

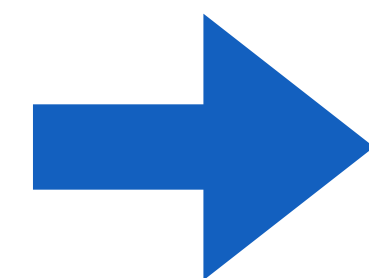
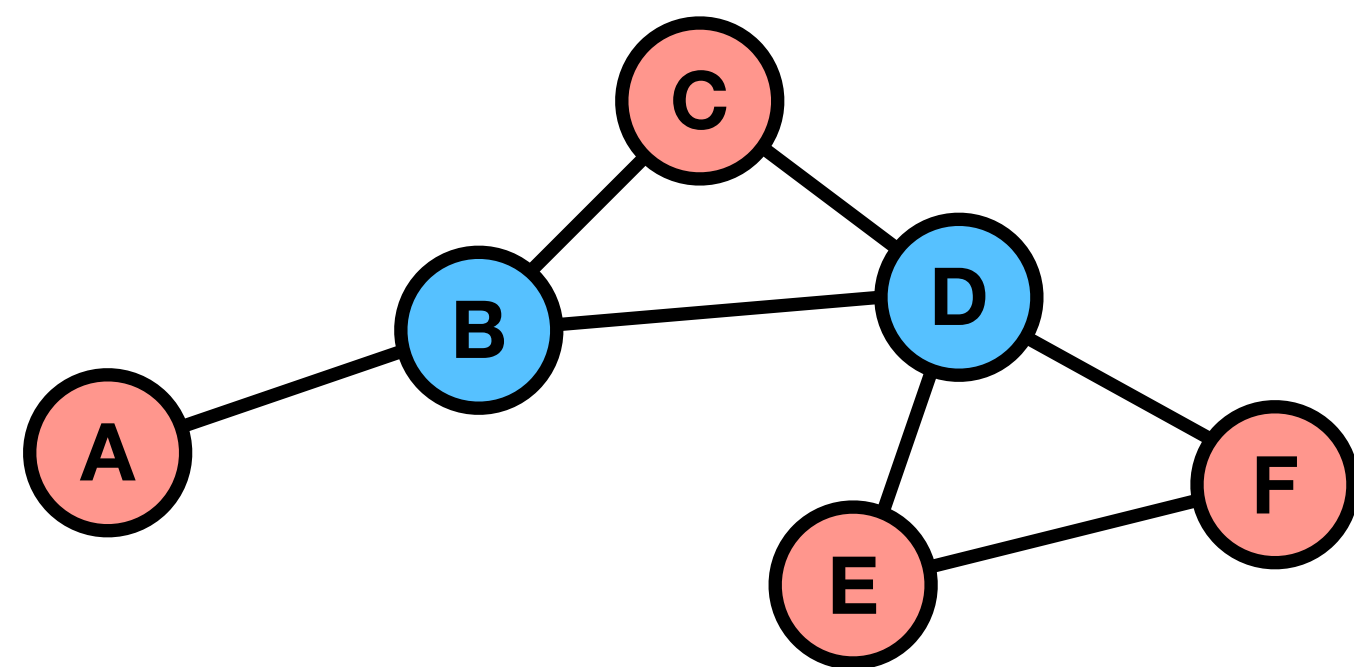
- Adjacency matrices can be **large and sparse**
- Number of parameters scales with  $O(|V|)$



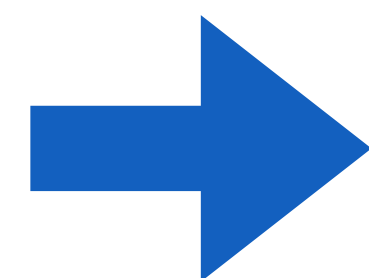
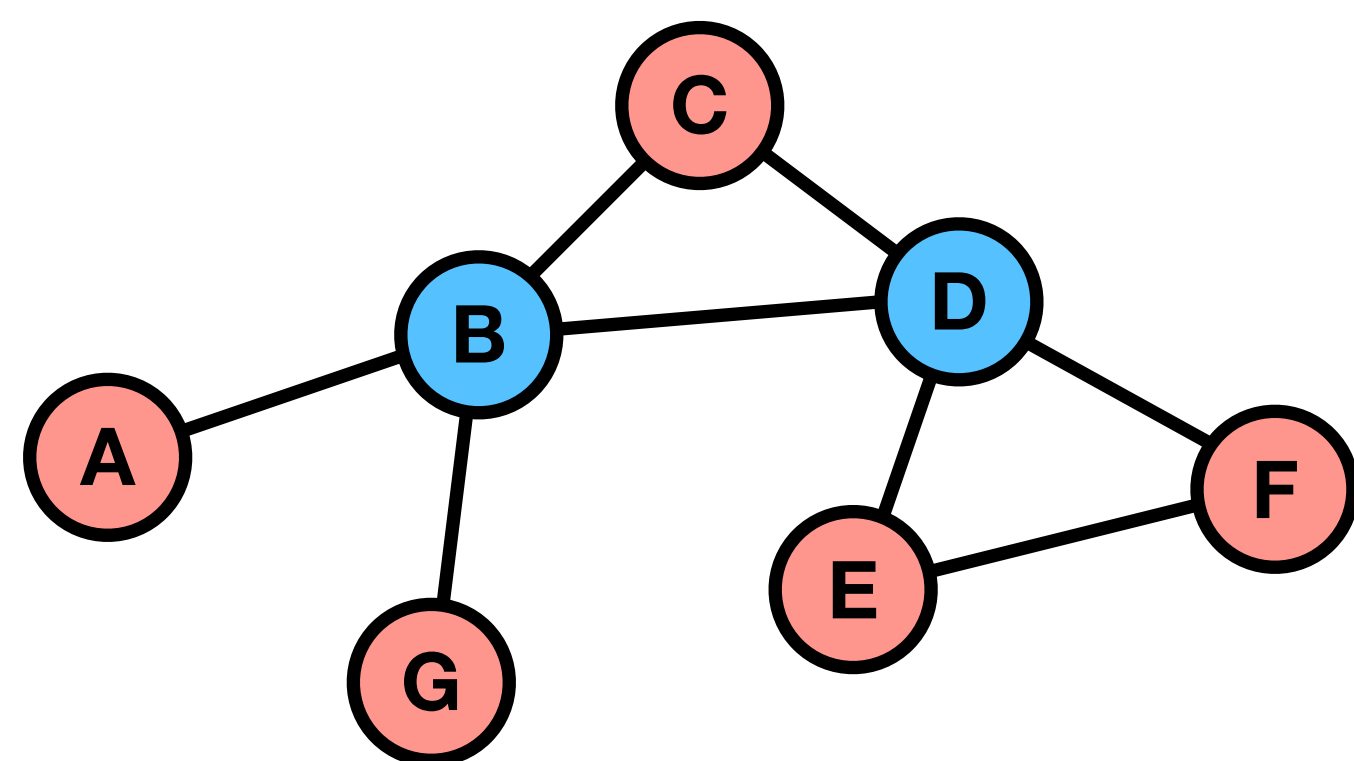
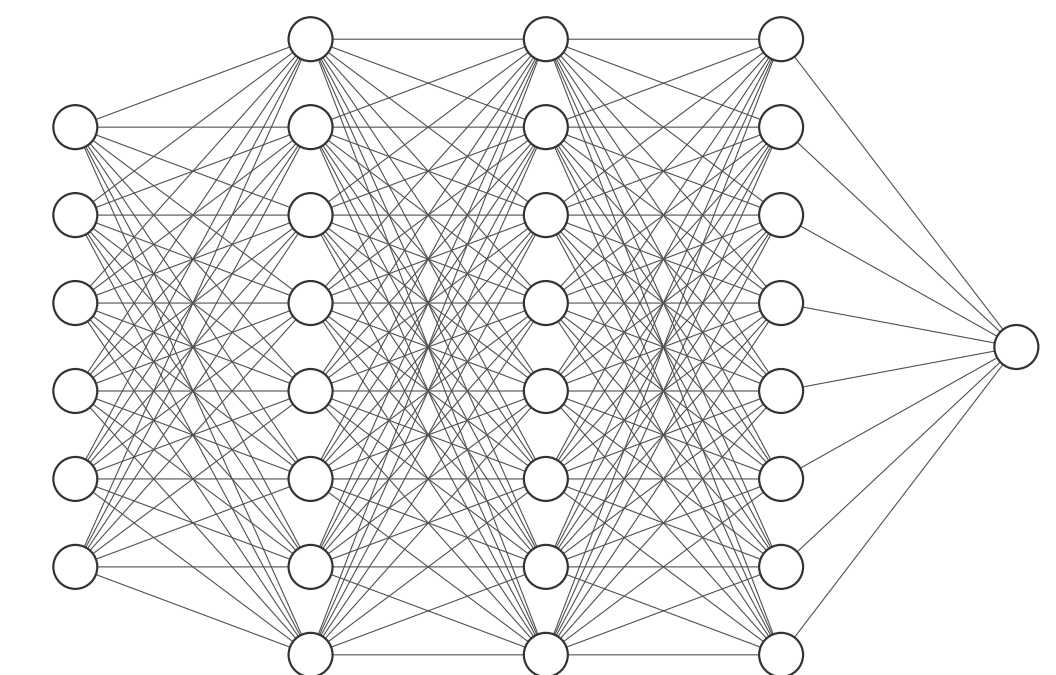
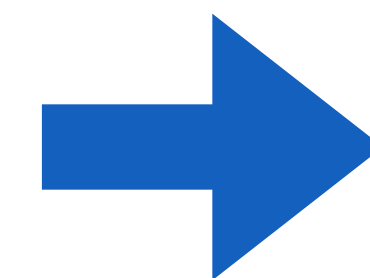
# A NAIVE APPROACH

Problem #2:

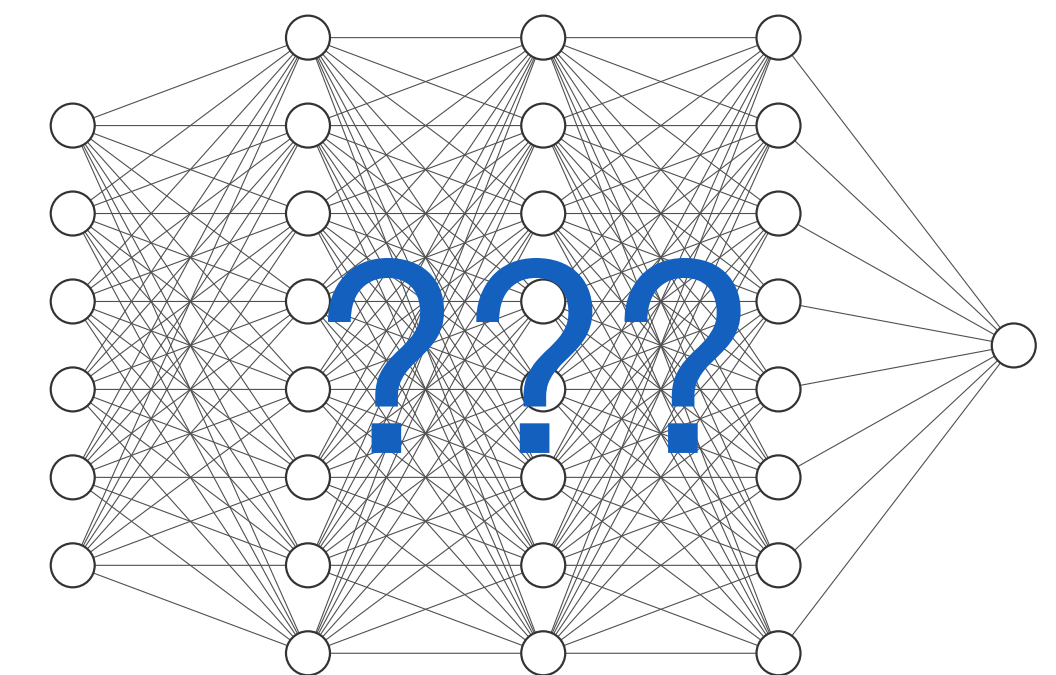
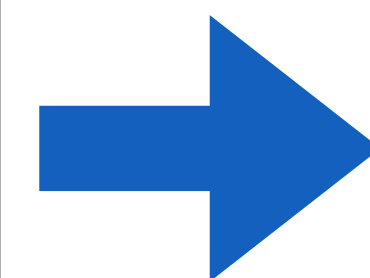
- Graphs may have different sizes
- MLPs cannot process unseen graph configurations ("transductive")



$$(A, F) = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$



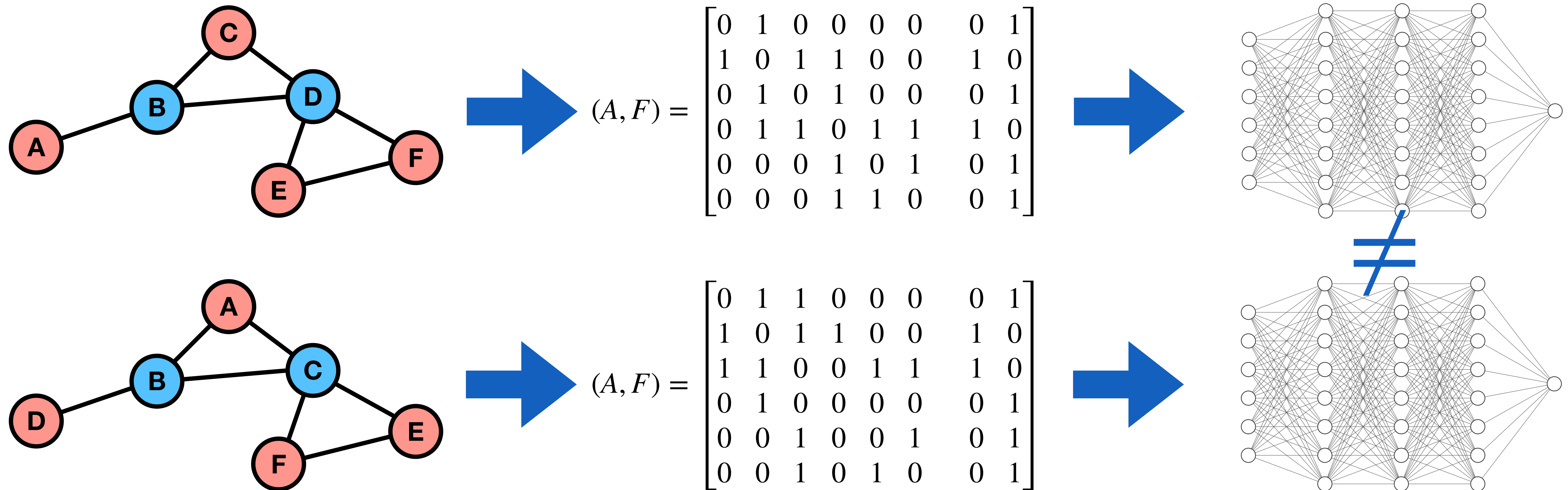
$$(A, F) = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$



# A NAIVE APPROACH

Problem #3:

- There exist  $|V|!$  permutations of graph orders but no canonical ordering
- MLPs are not permutation equivariant



## A NAIVE APPROACH

- Problems
  - Number of parameters scales with  $O(|V|)$
  - MLPs cannot process unseen graph configurations
  - MLPs are not permutation equivariant

**We need to design a neural network  
that is permutation equivariant**

# REMINDER: WEEK 4

MDL - GROUPS AND GROUP-EQUIVARIANT DL

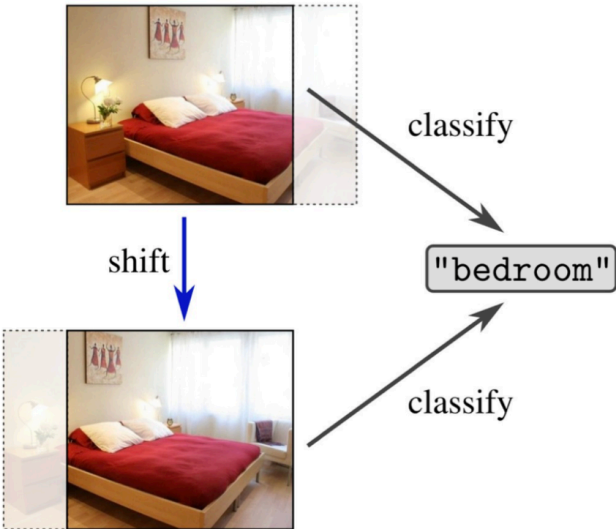
25 / 50



## INVARIANCE VS. EQUIVARIANCE

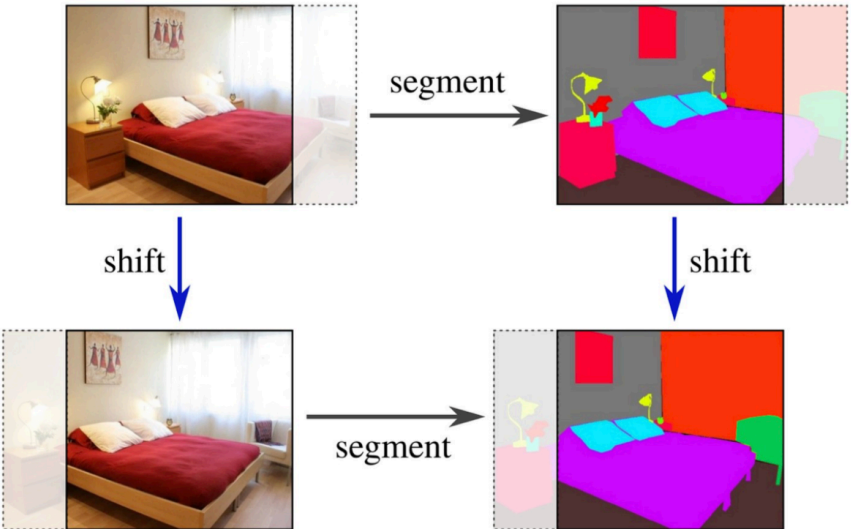
🤔 How do we include this information into our model architectures?

*invariant image classification*



The diagram shows two images of a bedroom. The top image is the original, and the bottom image is a shifted version. Both images are processed by a 'classify' step, resulting in the label '"bedroom"'. A 'shift' arrow points from the top image to the bottom image.

*equivariant image segmentation*



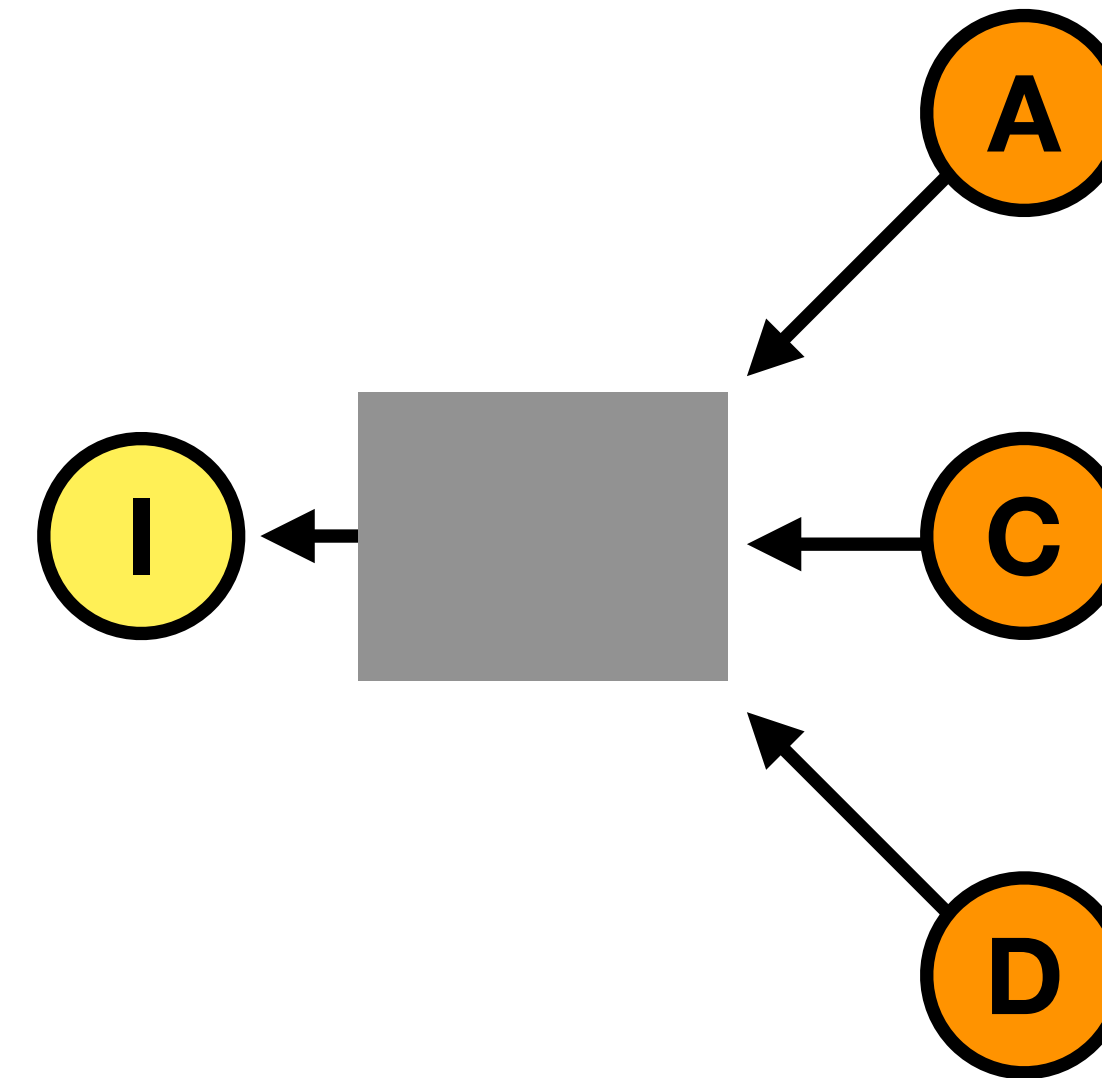
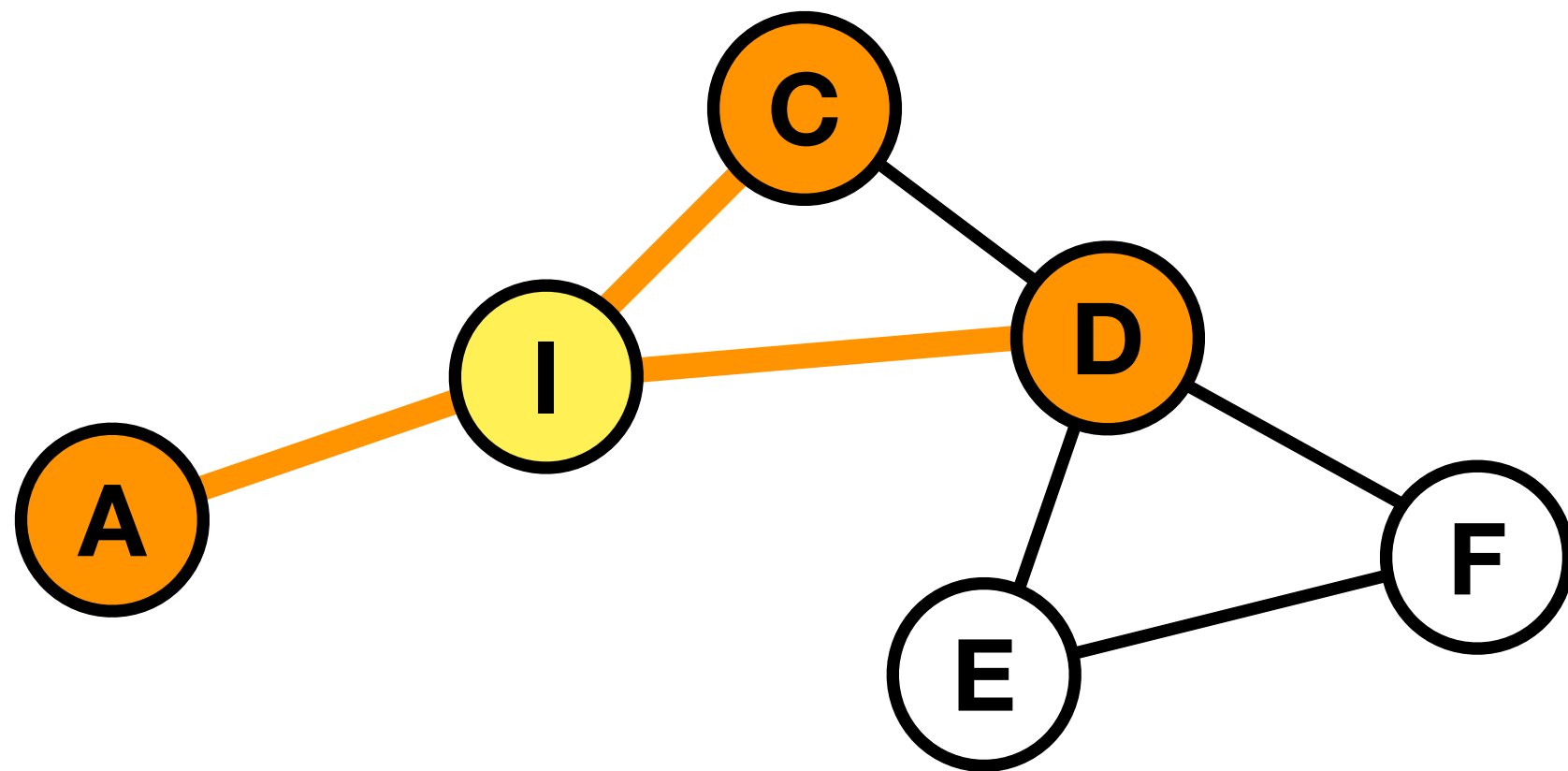
The diagram shows two images of a bedroom. The top image is the original, and the bottom image is a shifted version. Both images are processed by a 'segment' step, resulting in segmented images where different parts of the room are colored differently (e.g., red for bed, blue for wall). A 'shift' arrow points from the top image to the bottom image.

*Intuitively, invariance loses information while equivariance preserves it.*

[1] [https://www.sci.unich.it/geodeep2022/slides/Groups\\_Representations\\_and\\_Equivariance.pdf](https://www.sci.unich.it/geodeep2022/slides/Groups_Representations_and_Equivariance.pdf)

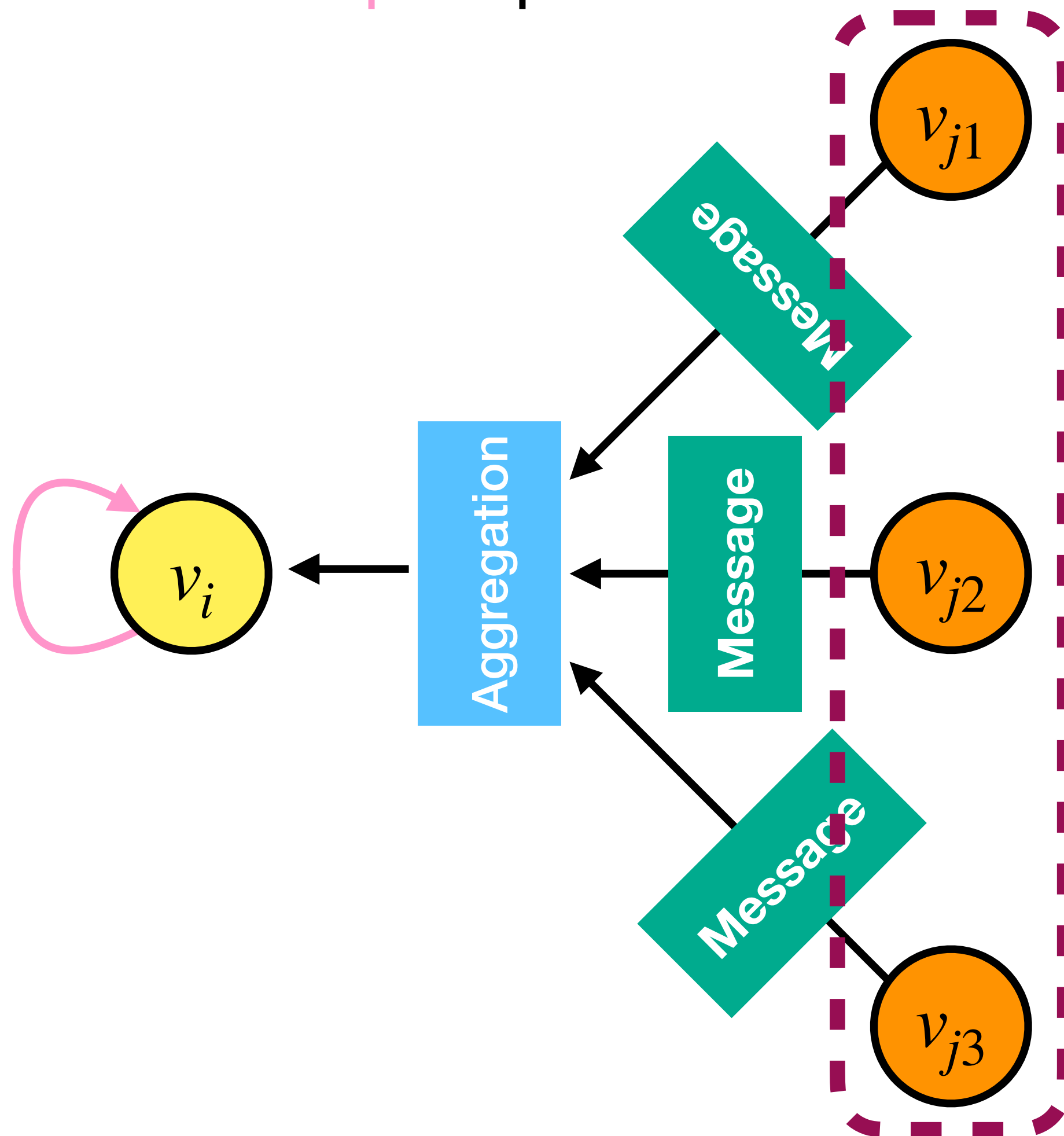
# GRAPH CONVOLUTIONAL NEURAL NETWORKS

- Step 1: determine node computation graph based on graph structures
- Step 2: propagate and transform information



# GRAPH CONVOLUTIONAL NEURAL NETWORKS

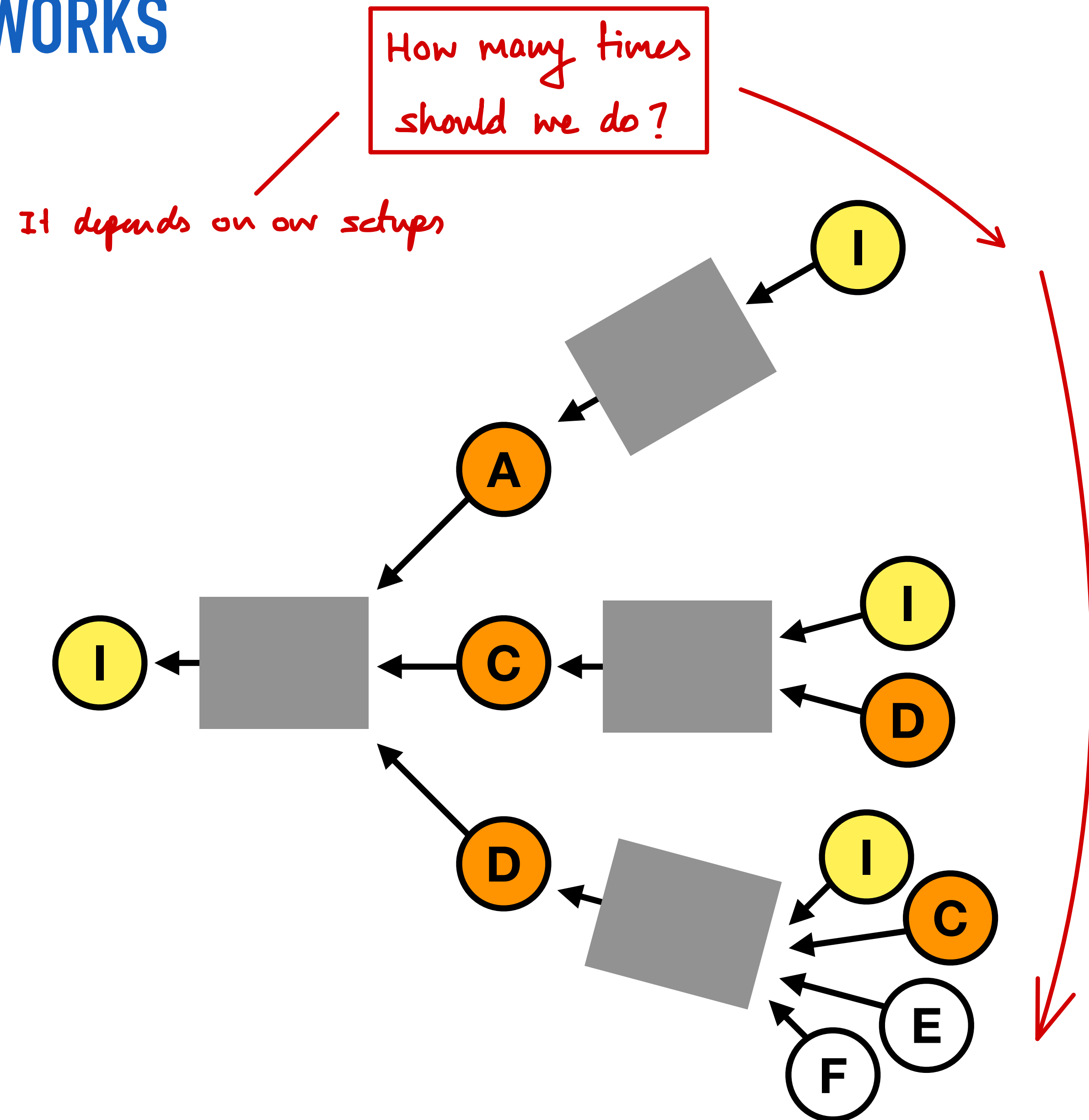
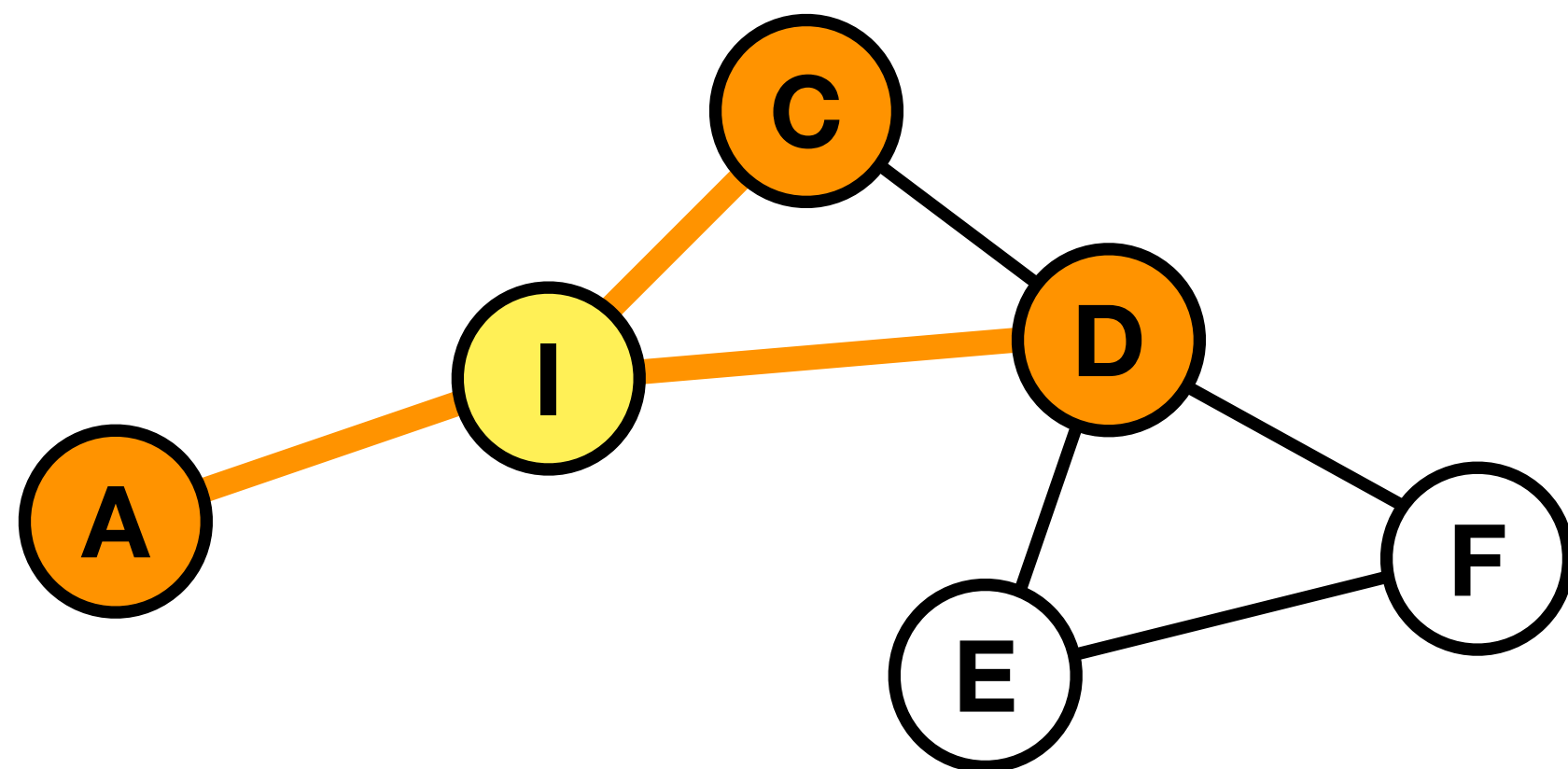
- Message passing and subsequent aggregation of neighboring nodes
- Self-loop to preserve content of updated node



$$v_i^{(l+1)} = \sigma \left( \sum_{j \in \mathcal{N}_i} \underbrace{\frac{1}{c_i}}_{\text{normalize}} W^{(l)} v_j^{(l)} + W_0^{(l)} v_i^{(l)} \right)$$

# GRAPH CONVOLUTIONAL NEURAL NETWORKS

- Multiple layers can be stacked



# AGGREGATION FUNCTION

- With increasing number of graph convolutional layers the receptive field grows exponentially
- Typical pooling functions:
  - $Mean(\cdot)$
  - $Sum(\cdot)$
  - $Max(\cdot)$
- Advanced network architectures use attention mechanisms

# REMINDER: WEEK 4

MDL - DL ON SETS, SEQUENCES, TUPLES

52 / 142



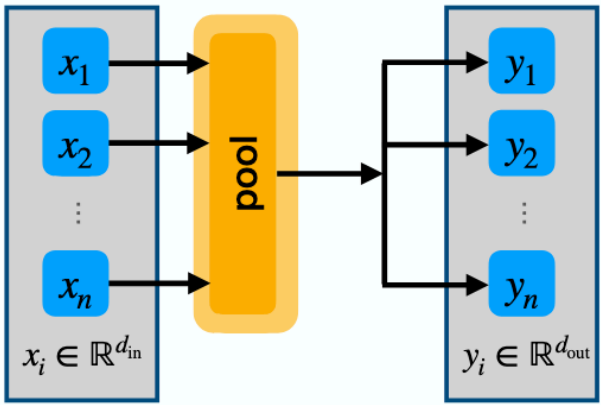
## POOL AND DISTRIBUTE

**Pool and Distribute**

1. Pool inputs to single vector (e.g. using mean/max pooling)
2. Copy pooled vector for each element

Optional

- Apply MLP after pool / before distribute
- Add outputs to
  - residual connections of  $x$
  - or element-wise operations on  $x$



The diagram illustrates the 'Pool and Distribute' operation. On the left, a vertical stack of input nodes  $x_1, x_2, \dots, x_n$  is shown, with the label  $x_i \in \mathbb{R}^{d_{in}}$  at the bottom. Arrows from these nodes point to a central yellow box labeled 'pool'. An arrow from the 'pool' box points to a vertical stack of output nodes  $y_1, y_2, \dots, y_n$  on the right, with the label  $y_i \in \mathbb{R}^{d_{out}}$  at the bottom. This represents a global pooling operation followed by a broadcast (distribute) to each output node.

|       | $y_1$ | $y_2$ | ... | $y_n$ |
|-------|-------|-------|-----|-------|
| $x_1$ | 1     | 1     | 1   | 1     |
| $x_2$ | 1     | 1     | 1   | 1     |
| ...   | 1     | 1     | 1   | 1     |
| $x_n$ | 1     | 1     | 1   | 1     |

Information Flow

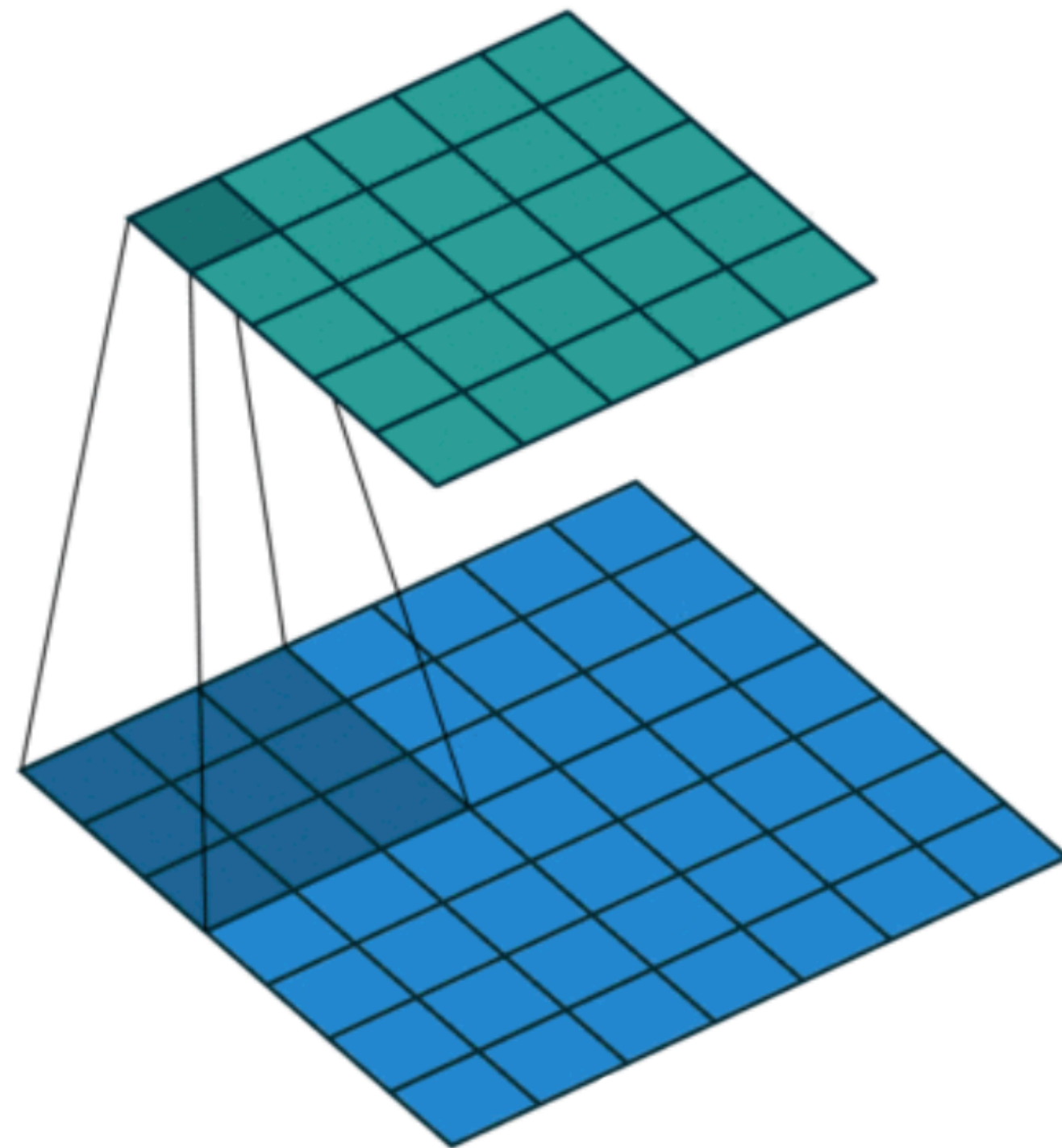
# GNN SUBSUME TRANSFORMER

**Transformers are graph neural networks -  
To be shown in the homework**

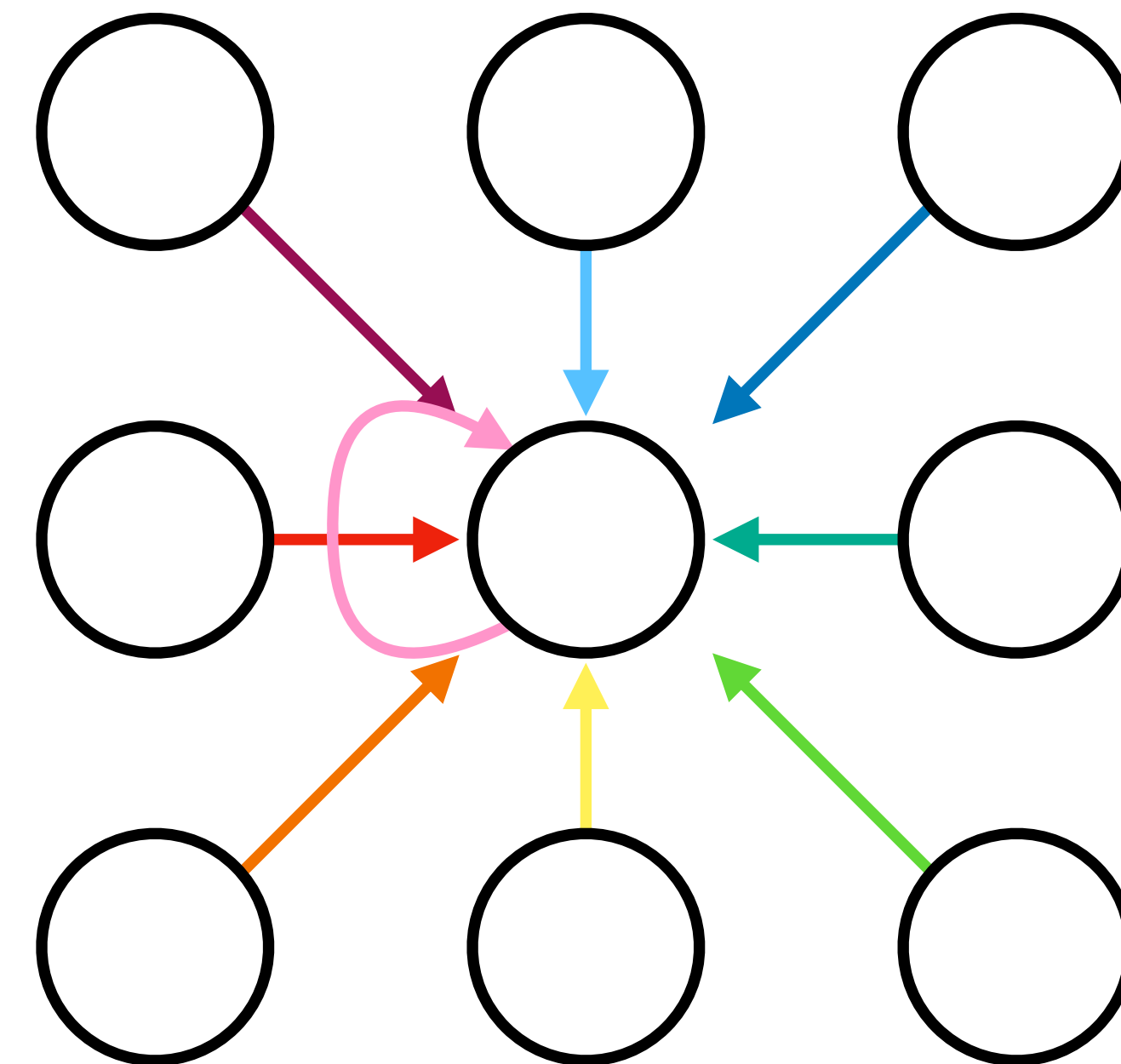
# GNNS SUBSUME CNNs

Images and sequences can be seen as special cases of graphs

CNNs can be seen as a special GNN with fixed neighborhood size and ordering



**3x3 convolution**

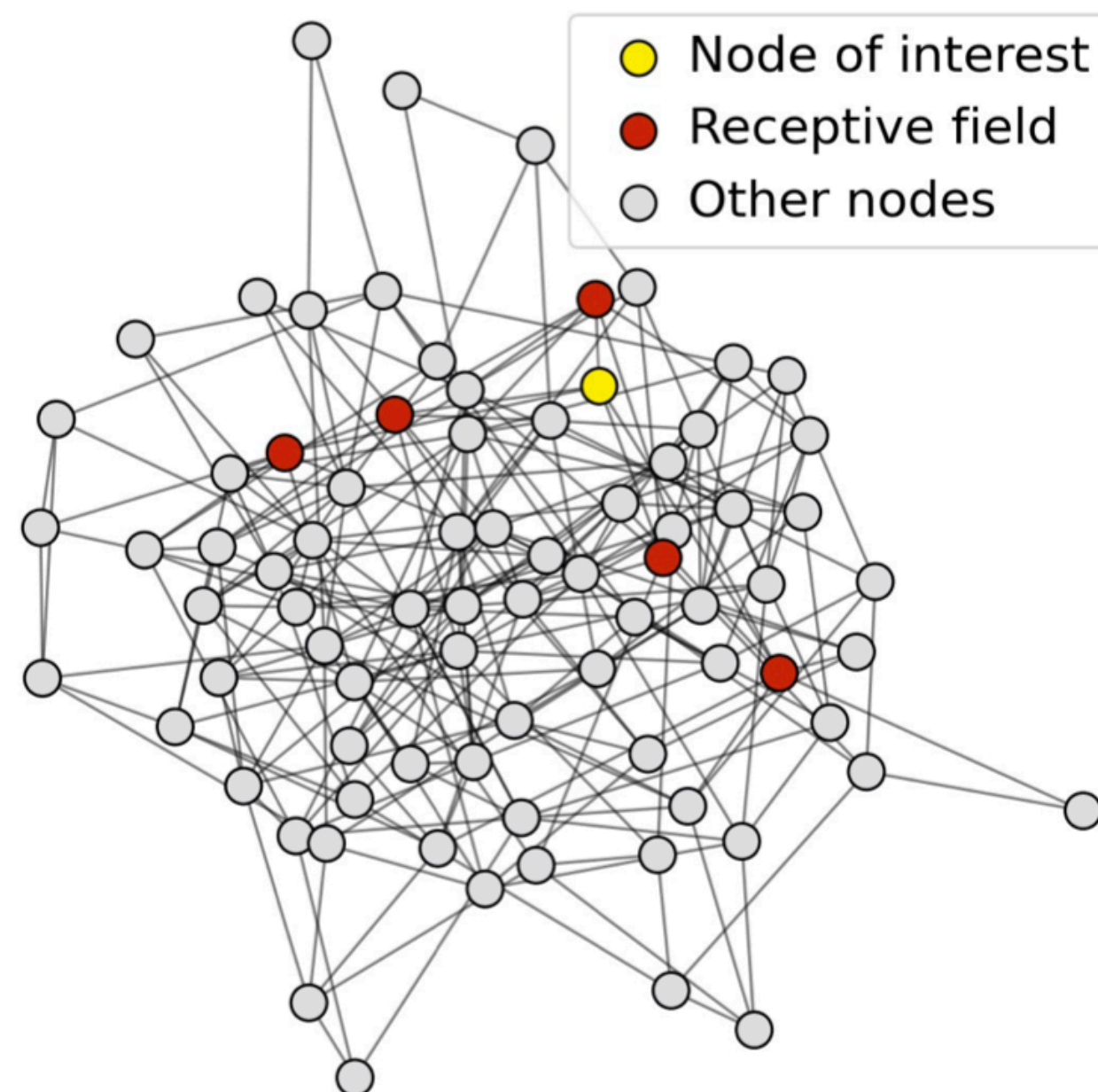


**3x3 convolution as graph**

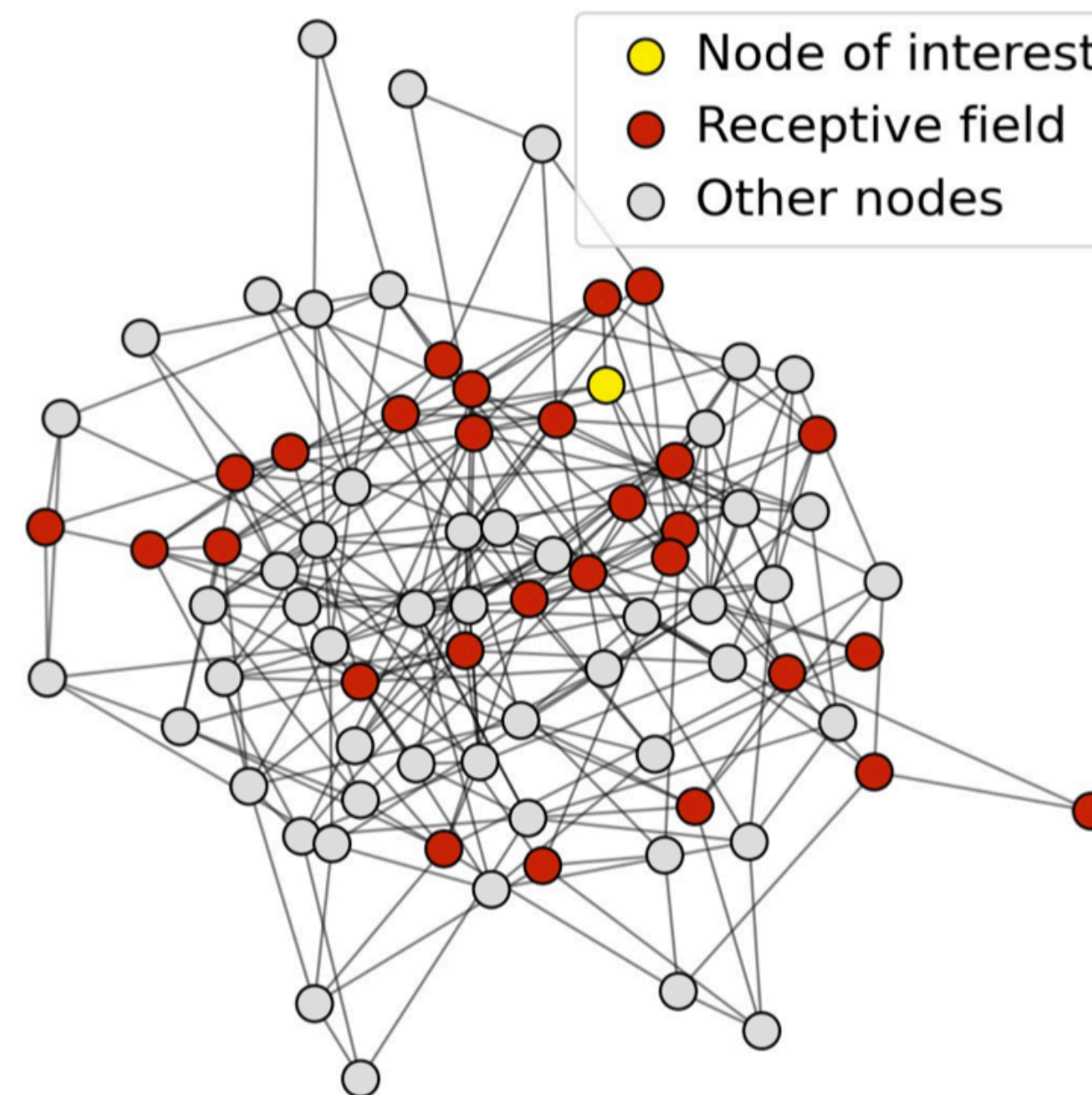
# GRAPH OVERSMOOTHING

With increasing number of graph convolutional layers the receptive field grows exponentially

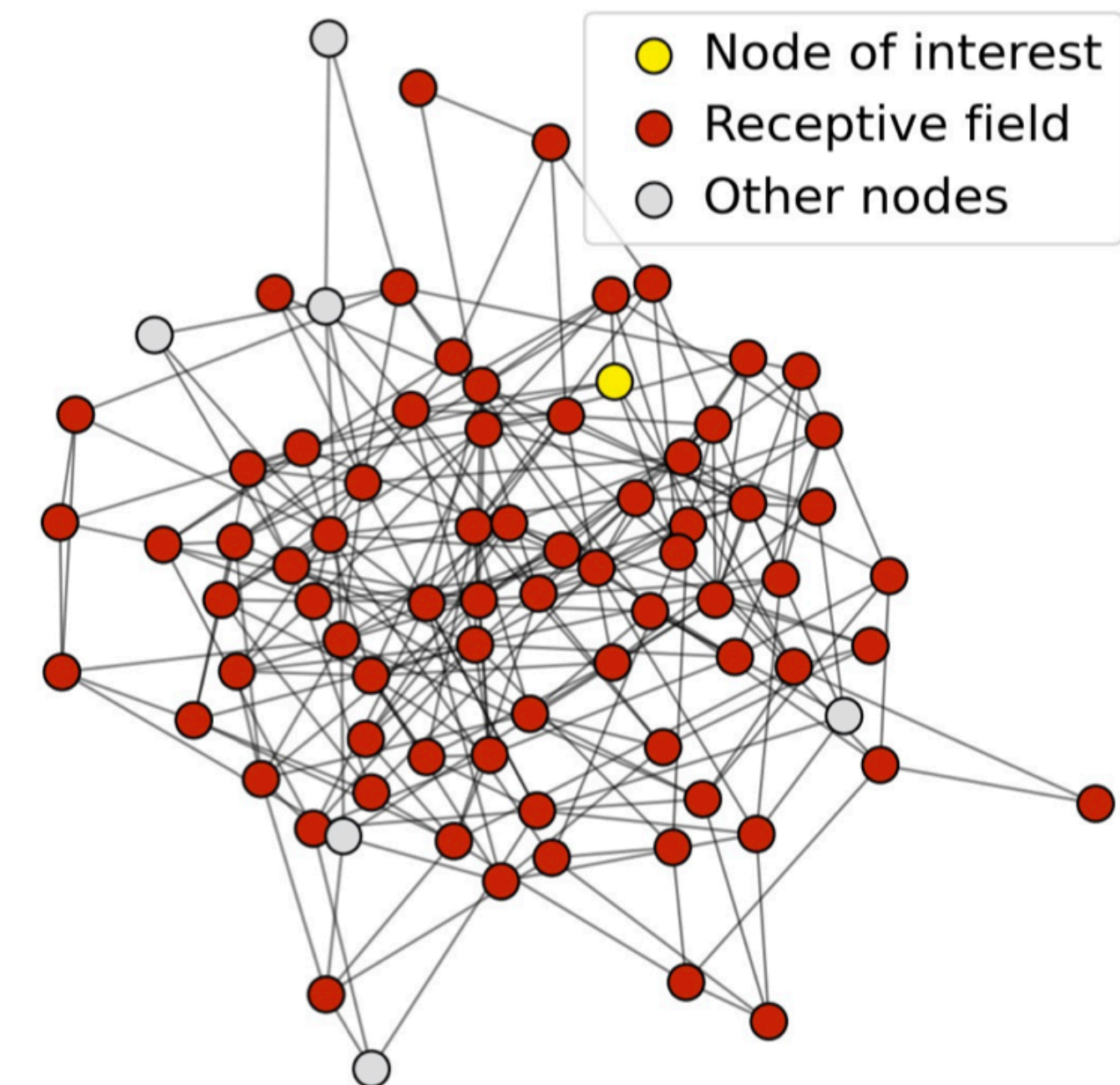
**Receptive field for  
1-layer GNN**



**Receptive field for  
2-layer GNN**



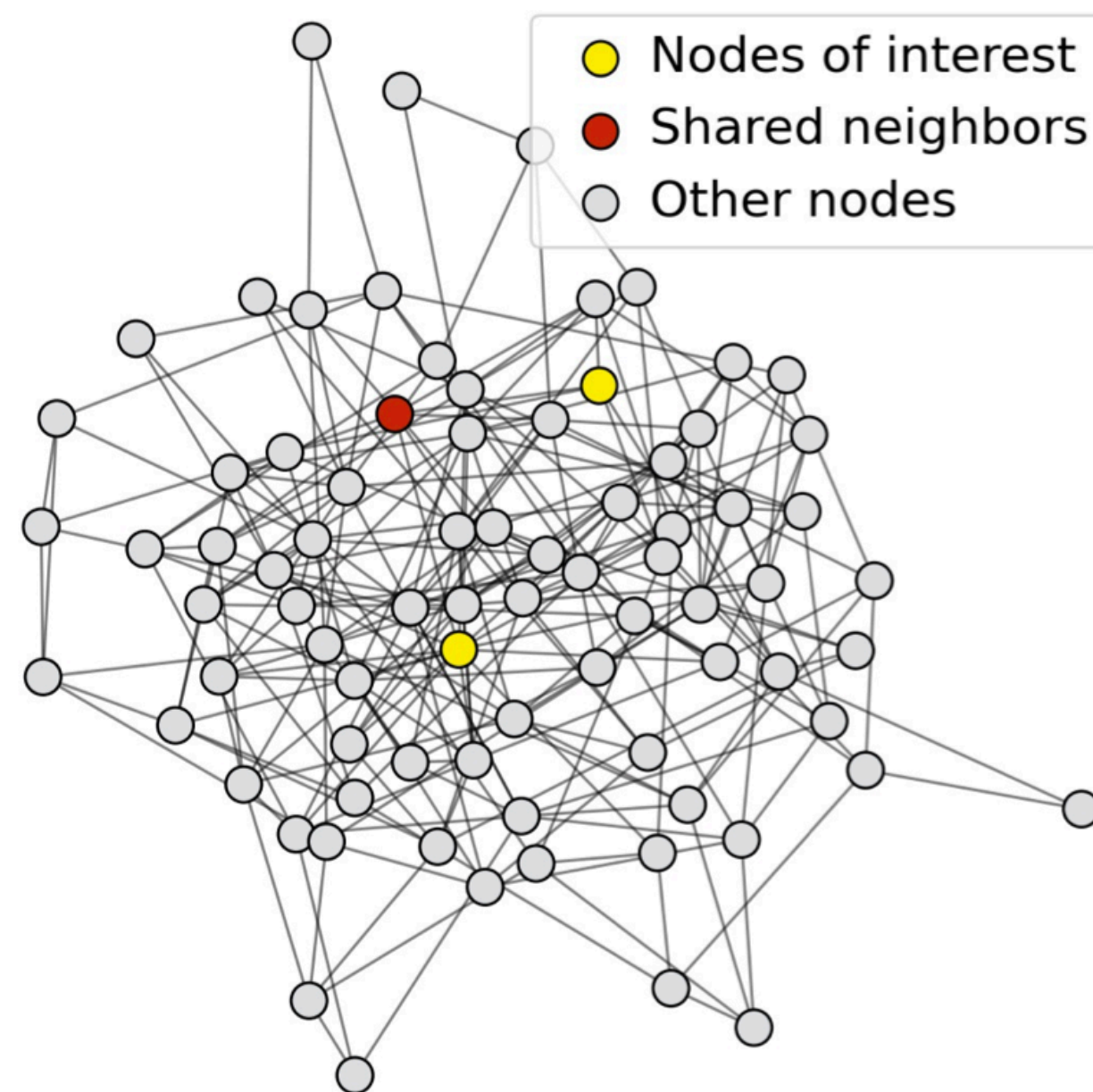
**Receptive field for  
3-layer GNN**



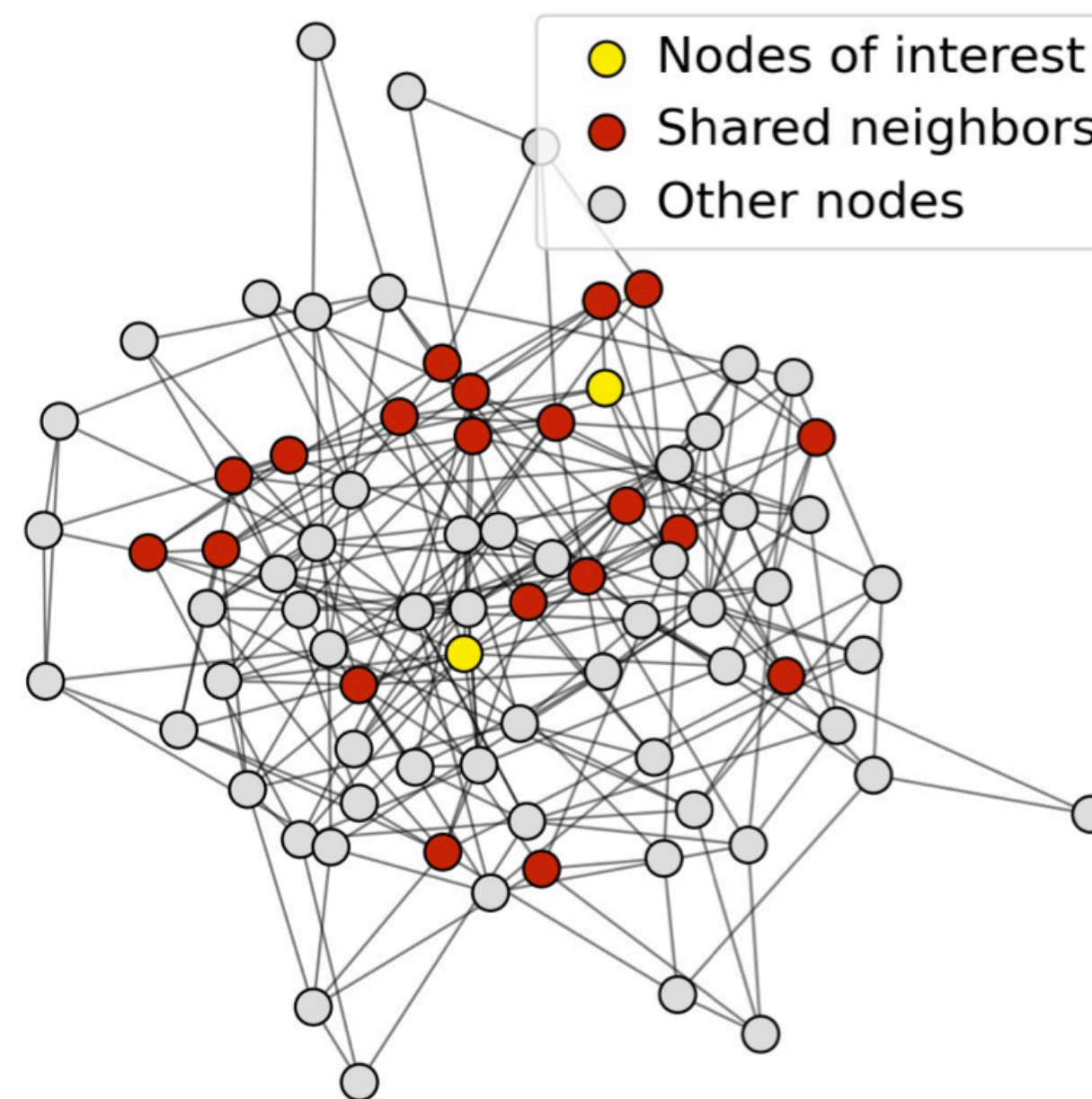
# GRAPH OVERSMOOTHING

As the receptive fields of all nodes overlap, they become highly similar, leading to degenerate node embeddings

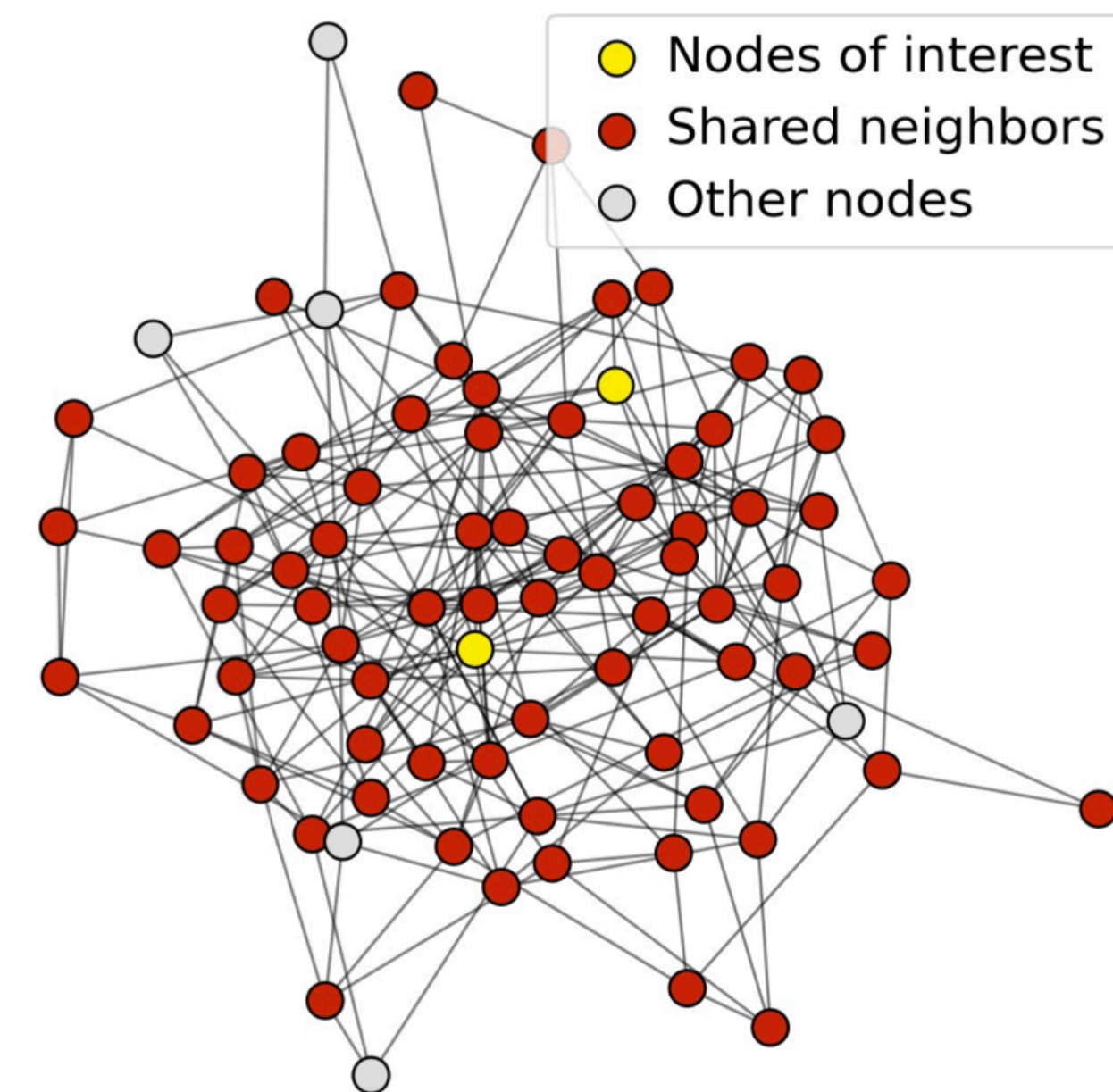
**1-hop neighbor overlap**  
**Only 1 node**



**2-hop neighbor overlap**  
**About 20 nodes**



**3-hop neighbor overlap**  
**Almost all the nodes!**



# SUMMARY

## Strengths of graph deep learning:

- Explicit inductive bias
- Light-weight
- Parameter-efficient
- Flexible with missing modalities

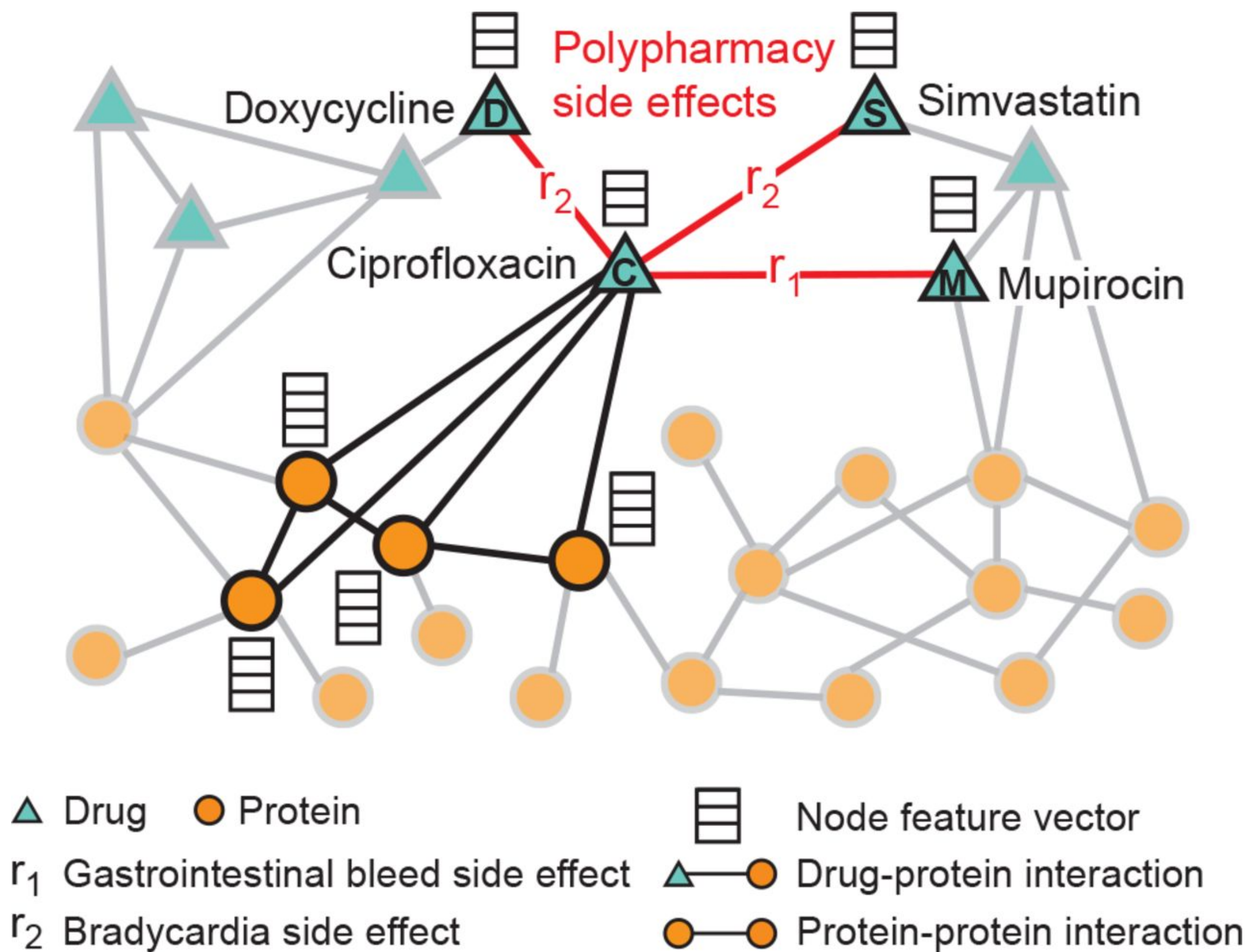
## Weaknesses:

- Dependent on graph quality
- Limited long-range dependency modeling
- Cumbersome implementation
- Lack of scalability

# GRAPHS IN MULTIMODAL DEEP LEARNING

- Graphs are commonly found in **multimodal deep learning**
  - Graph as **input** of a multimodal pipeline
  - Graph as **output** of a multimodal pipeline
  - Graph as **structure to relate** multimodal data embeddings

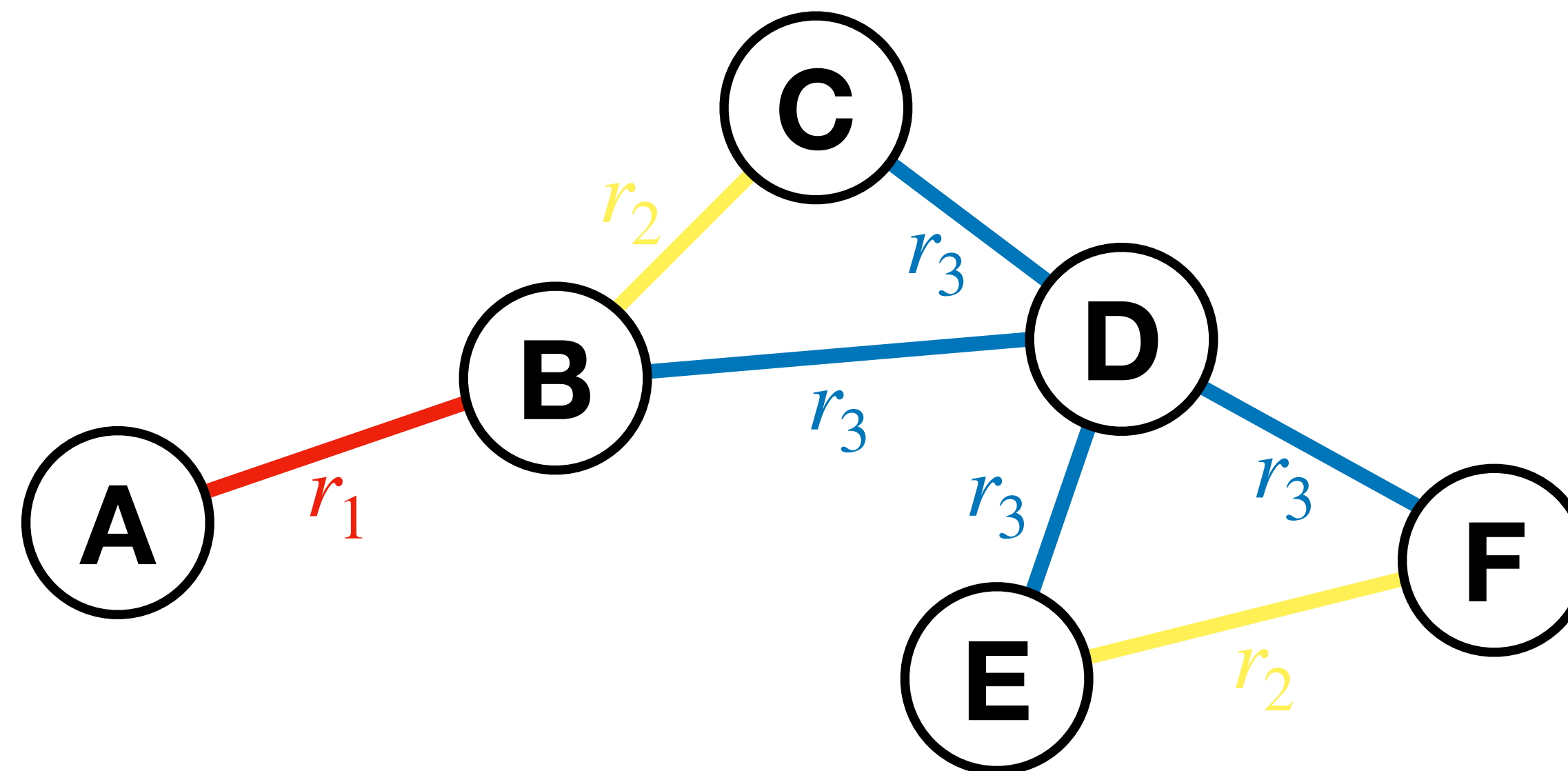
# HETEROGENEOUS GRAPHS



Drug interaction graph

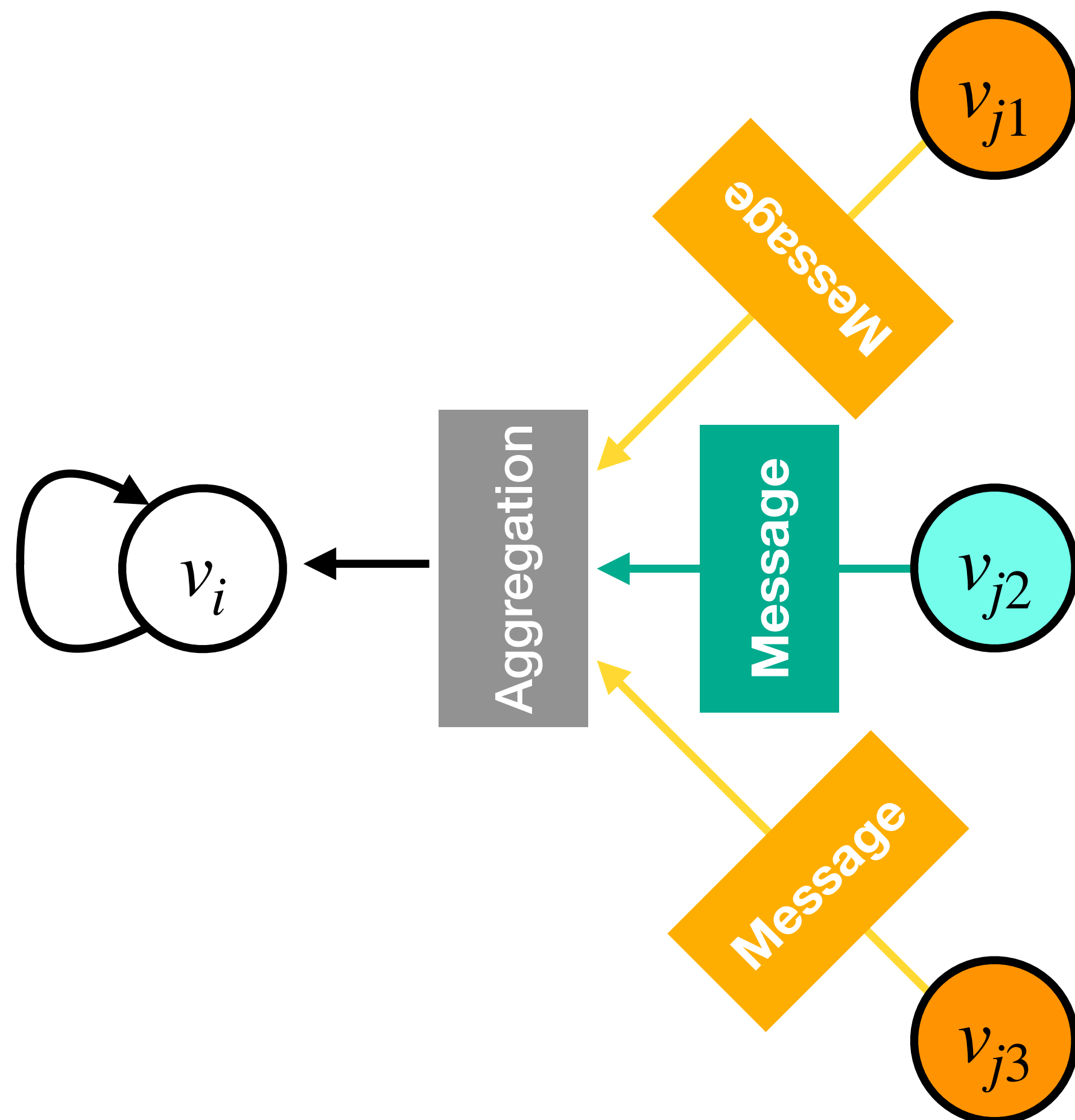
# HETEROGENEOUS GRAPHS

Each nodes have a type  $\tau(v)$ , each edge has a type  $\phi(v_i, v_j)$ ,  
resulting in a relation type for each edge  $r(v_i, v_j) = (\tau(v_i), \phi(v_i, v_j), \tau(v_j))$



# RELATIONAL GRAPH CONVOLUTIONAL NEURAL NETWORKS

Use of different message weight matrices depending on the type of relation



$$v_i^{(l+1)} = \sigma \left( \sum_{r \in R} \sum_{j \in \mathcal{N}_i^r} \frac{1}{c_{i,r}} W_r^{(l)} v_j^{(l)} + W_0^{(l)} v_i^{(l)} \right)$$

# RELATIONAL GRAPH CONVOLUTIONAL NEURAL NETWORKS

- Rapid growth of parameters with number of relations
  - Overfitting becomes an issue
- Potential mitigation strategies
  - Use of sparse (block diagonal) weight matrices
  - Sharing of weights across different message matrices

## FURTHER MATERIAL

- TUM lecture Machine Learning for Graphs and Sequential Data (IN2323)
- Stanford Machine Learning with Graphs lecture by Jure Leskovec (<https://web.stanford.edu/class/cs224w>)
- Graph Representation Learning Book by William L. Hamilton ([https://www.cs.mcgill.ca/~wlh/grl\\_book](https://www.cs.mcgill.ca/~wlh/grl_book))